Microservices Solution Doc — Evolution Series

Java · Spring Boot · Kafka · PostgreSQL · Cassandra · Redis · CI/CD · Canary · Blue-Green · Logs · Metrics

Architecture Deep Dive

Contents

Tech Stack: Java • Spring Boot • Kafka • PostgreSQL • Cassandra • Redis • CI/CD • Canary • Blue-Green • Logs • Metrics . . . 4
Design Architecture Journey — Series Overview . . . 4
Part I — Monolithic Architecture . . . 4
1. Definition . . . 4
2. Hands-On: Building a Monolithic E-Commerce App (Spring Boot) . . . 4
3. Real-Time Usage . . . 8
4. Architectural Flow Diagram . . . 9
Coming Next — Part II . . . 9
Part II — Modular Monolith → Microservices / Event-Driven → Serverless . . . 11
Q1: Modular Monolith Architecture . . . 11
Definition . . . 11
Hands-On: Refactoring the E-Commerce Monolith into Modules (Spring Modulith) 11
Real-Time Usage . . . 13
Architectural Flow Diagram . . . 13
Q2: Microservices & Event-Driven Microservices . . . 13
Definition . . . 13
Hands-On: Splitting the Monolith into 3 Microservices + Kafka . . . 14
Real-Time Usage . . . 19
Architectural Flow Diagram . . . 19
Q3: Serverless Architecture — Designing for Millions of Requests . . . 20
Definition . . . 20
Hands-On: Serverless Order-Processing Pipeline (Java + AWS) . . . 20
Real-Time Usage . . . 23
Architectural Flow Diagram . . . 23
Coming Next — Part III . . . 23
Part III — Decomposition • Communication • Data Management • Caching • Deployment . . . 25
Q1: Microservices Decomposition . . . 25
Definition . . . 25
Hands-On: Applying DDD + Strangler Fig to the E-Commerce Monolith . . . 25
Real-Time Usage . . . 26
Architectural Flow Diagram . . . 26
Q2: Microservices Communication (Sync / Async, API Gateway, RestTemplate, Feign, gRPC) . . . 26
Definition . . . 26
Designing HTTP-Based RESTful APIs for Microservices . . . 29
Hands-On . . . 29
Real-Time Usage . . . 33

Q3: Microservices Data Management (Databases, CQRS, Event Sourcing, Distributed Transactions) . . . 33
Definition . . . 33
Hands-On: CQRS + Event Sourcing for Orders (PostgreSQL write side, Cassandra read side) . . . 34
Real-Time Usage . . . 37
Architectural Flow Diagram . . . 37
Q4: Microservices Distributed Caching (Caffeine, Redis, Eviction Policies, @CachePut) 37
Definition . . . 37
Hands-On: Two-Level Cache (Caffeine L1 + Redis L2) for Product Lookups . . . 37
Real-Time Usage . . . 44
Architectural Flow Diagram . . . 44
Q5: Microservices Deployment — Containers, Orchestrators, CI/CD, Canary, Blue-Green . . . 44
Definition . . . 44
Hands-On . . . 46
Real-Time Usage . . . 48
Architectural Flow Diagram . . . 48
Coming Next — Part IV . . . 49
Part IV — 3-Tier • API Gateway/BFF • Registry/Aggregator • Broker • Sharding • Event-Driven (Outbox) • Caching • Containers/CI-CD • Resilience4j . . . 50
Q1: 3-Tier Architecture . . . 50
Definition . . . 50
Hands-On: Classic 3-Tier Deployment (physically separated) . . . 50
Real-Time Usage . . . 51
Architectural Flow Diagram . . . 51
Q2: Microservices Architectures (System Overview) . . . 51
Definition — Recap . . . 51
Hands-On: Wiring the Whole System Together (docker-compose) . . . 51
Real-Time Usage . . . 52
Architectural Flow Diagram . . . 52
Q3: Microservices Architecture — API Gateways (Deep Dive) . . . 53
Definition . . . 53
Hands-On: Spring Cloud Gateway with JWT Auth + Rate Limiting . . . 54
Real-Time Usage . . . 55
Architectural Flow Diagram . . . 55
Q4: Microservices Architecture — API Gateway BFF (Backend For Frontend) . . . 55
Definition . . . 55
Hands-On: mobile-bff vs web-bff aggregating the same data differently . . . 55
Real-Time Usage . . . 58
Architectural Flow Diagram . . . 58
Q5: Microservices Architecture — Service Aggregator / Registry Patterns . . . 58
Definition . . . 58
Hands-On: Eureka Registry + a Dedicated Aggregator Service . . . 59
Real-Time Usage . . . 61
Architectural Flow Diagram . . . 61
Q6: Microservices Architecture — Message Broker . . . 61
Definition . . . 61
Hands-On: RabbitMQ with Spring AMQP (complementing the Kafka setup from Part II) . . . 63
Real-Time Usage . . . 64
Architectural Flow Diagram . . . 64
Q7: Microservices Architecture — Database Sharding (Cassandra) . . . 64
Definition . . . 64

Hands-On: Designing a Sharded Order Table . . . 65
Real-Time Usage . . . 66
Architectural Flow Diagram . . . 66
Q8: Event-Driven Microservices Architecture (Recap + Transactional Outbox) . . . 66
Definition — Recap . . . 66
Hands-On . . . 68
Real-Time Usage . . . 69
Architectural Flow Diagram . . . 69
Q9: Microservices Distributed Caching (Recap + Redis Cluster / Distributed Lock) . 69
Definition — Recap . . . 69
Hands-On: Redis Cluster HA + Distributed Lock (Redisson) to prevent cache stampede . . . 69
Real-Time Usage . . . 70
Architectural Flow Diagram . . . 70
Q10: Microservices Containers and Orchestrators with CI/CD (Recap + Autoscaling & Helm) . . . 70
Definition — Recap . . . 70
Hands-On . . . 70
Real-Time Usage . . . 72
Architectural Flow Diagram . . . 72
Q11: Microservices Resilience4j — Circuit Breaker, Retry, Rate Limiter, Bulkhead . . 72
Definition . . . 72
Hands-On: Full Stack — Circuit Breaker + Retry + Rate Limiter + Bulkhead on the Feign call to `product-service` . . . 73
Hands-On: Applying Resilience4j at the API Gateway Level (`apigateway.yaml`) . . . 75
Real-Time Usage . . . 80
Architectural Flow Diagram . . . 80
Coming Next — Part V . . . 80

Tech Stack: Java • Spring Boot • Kafka • PostgreSQL • Cassandra • Redis • CI/CD • Canary • Blue-Green • Logs • Metrics

Design Architecture Journey — Series Overview

This series walks through the full evolution of backend architecture, in order:

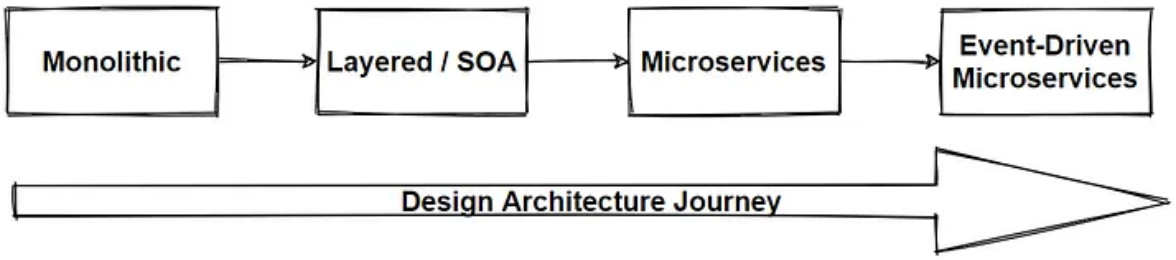


Figure 1: Design Architecture Journey — Monolithic to Layered/SOA to Microservices to Event-Driven Microservices

Each stage below builds on the limitations of the previous one — a monolith's coupling problems motivate the modular monolith and layered/SOA styles, whose remaining deployment/scaling coupling motivates microservices, whose synchronous coupling motivates event-driven microservices.

Part I — Monolithic Architecture

1. Definition

A **Monolithic Architecture** is a software design pattern where an entire application is built and deployed as a **single, unified unit**. All the functional modules — UI, business logic, data access, and integration layers — live in one codebase, run in one process, and are packaged into a single deployable artifact (a WAR/JAR file).

Key characteristics:

Aspect	Description
Codebase	Single repository, single build
Deployment	One artifact (JAR/WAR) deployed to one or more identical instances
Database	Usually one shared relational database
Scaling	Scales the *entire* app horizontally (you can't scale just the "Order" module alone)
Communication	In-process method calls (no network hop between modules)
Technology	Typically single language/framework across the app

---

2. Hands-On: Building a Monolithic E-Commerce App (Spring Boot)

We'll build a simple **E-Commerce Monolith** with three logical modules — `User`, `Product`, `Order` — packaged as ONE Spring Boot application, backed by ONE PostgreSQL database.

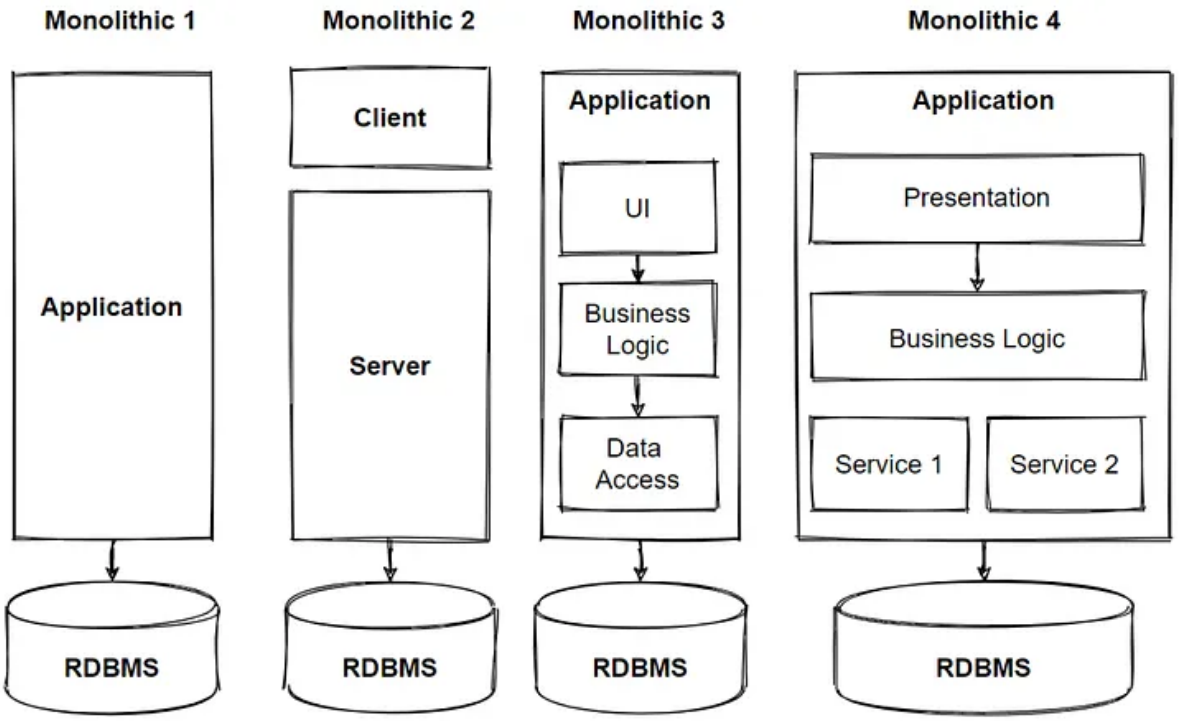


Figure 2: Monolithic Architecture Variants — from a single Application block, to Client/Server, to a layered UI/Business Logic/Data Access app, to a Presentation/Business Logic app with embedded services — all backed by one RDBMS

2.1 Project Structure

```
ecommerce-monolith/
 ├── src/main/java/com/example/ecommerce/
 │    ├── EcommerceMonolithApplication.java
 │    ├── user/
 │    │    ├── User.java
 │    │    ├── UserRepository.java
 │    │    ├── UserService.java
 │    │    └── UserController.java
 │    ├── product/
 │    │    ├── Product.java
 │    │    ├── ProductRepository.java
 │    │    ├── ProductService.java
 │    │    └── ProductController.java
 │    └── order/
 │         ├── Order.java
 │         ├── OrderRepository.java
 │         ├── OrderService.java
 │         └── OrderController.java
 ├── src/main/resources/application.yml
 └── pom.xml
```

Even though `user`, `product`, and `order` look “modular” by package, they are **NOT independently deployable** — this is the defining trait of a monolith.

```xml
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
</dependencies>
```

2.2 `pom.xml` (key dependencies)

```yaml
server:
  port: 8080

spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ecommerce_db
    username: postgres
    password: postgres
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

2.3 `application.yml` Notice: **one datasource** for all three modules — User, Product, and Order tables all live in `ecommerce_db`.

```java
package com.example.ecommerce.order;

import jakarta.persistence.*;
import java.math.BigDecimal;

@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long userId;
    private Long productId;
    private Integer quantity;
```

```
    private BigDecimal totalPrice;
    private String status; // CREATED, PAID, SHIPPED

    // getters and setters
}
```

2.4 Entity — `Order.java`

2.5 Service Layer — `OrderService.java` (In-Process Calls) This is the heart of monolithic behavior: `OrderService` directly calls `ProductService` and `UserService` as **Java method calls**, not network/API calls.

```
package com.example.ecommerce.order;

import com.example.ecommerce.product.ProductService;
import com.example.ecommerce.user.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class OrderService {

    @Autowired
    private OrderRepository orderRepository;

    @Autowired
    private ProductService productService; // direct in-JVM call

    @Autowired
    private UserService userService;        // direct in-JVM call

    public Order placeOrder(Long userId, Long productId, Integer qty) {
        userService.validateUser(userId);               // method call, not HTTP call
        var product = productService.reserveStock(productId, qty); // method call

        Order order = new Order();
        order.setUserId(userId);
        order.setProductId(productId);
        order.setQuantity(qty);
        order.setTotalPrice(product.getPrice().multiply(java.math.BigDecimal.valueOf(qty)));
        order.setStatus("CREATED");

        return orderRepository.save(order);
    }
}
```

```
package com.example.ecommerce.order;

import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/orders")
public class OrderController {

    private final OrderService orderService;
```

```
    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    @PostMapping
    public Order create(@RequestParam Long userId,
                        @RequestParam Long productId,
                        @RequestParam Integer qty) {
        return orderService.placeOrder(userId, productId, qty);
    }
}
```

2.6 Controller — `OrderController.java`

```
# Build the single artifact
mvn clean package

# Run it — ONE process handles Users, Products, AND Orders
java -jar target/ecommerce-monolith-0.0.1-SNAPSHOT.jar

# Scale it — you can only scale the WHOLE app, not just Orders
docker run -p 8081:8080 ecommerce-monolith:latest
docker run -p 8082:8080 ecommerce-monolith:latest
```

2.7 Run & Deploy Put both instances behind a load balancer (e.g., Nginx/HAProxy) — this is "scaling," but every replica still carries the full weight of User + Product + Order logic, even if only Orders is under heavy load.

---

3. Real-Time Usage

Scenario	Why Monolith Fits (or Fails)
Early-stage startup / MVP	Fast to build, one team, one deploy pipeline — ideal for validating a product idea quickly (e.g., early Amazon, early Etsy, early Shopify architectures).
Small internal tools	HR portals, inventory tools for a single department — low traffic, low complexity, no need for independent scaling.
Airline/Banking legacy cores	Many still run COBOL/Java monoliths for core transaction processing — stable, ACID-compliant, hard to justify a rewrite.
Where it breaks down	An e-commerce platform during a flash sale: `Order` traffic spikes 50x, but you're forced to scale `User` and `Product` code you don't need — wasting compute and slowing deployments (one bug in `Product` can crash `Order` and `User` too, since they share a JVM and DB connection pool).

Scenario	Why Monolith Fits (or Fails)
Deployment risk	A single typo in the `Product` module requires a full re-deploy and re-test of `User` and `Order` as well — release cycles slow down as the team/codebase grows ("nobody wants to touch the monolith").

This pain — **coupled scaling, coupled deployment, coupled failure domains** — is exactly the motivation for evolving toward Microservices in Part II.

---

4. Architectural Flow Diagram

Flow explanation: 1. Client sends an HTTP request → hits the Load Balancer. 2. Load Balancer routes to any identical monolith instance (all instances run 100% of the code). 3. Inside the JVM, `OrderController` → `OrderService` → directly invokes `UserService.validateUser()` and `ProductService.reserveStock()` as **plain Java method calls** (no network hop, no serialization). 4. All three services persist to the **same** PostgreSQL database/schema. 5. A single instance is a single point of change: redeploying means the whole JAR restarts, including User and Product functionality.

Coming Next — Part II

Step-by-step decomposition of this monolith into Microservices: splitting User, Product, and Order into independently deployable Spring Boot services, introducing **Kafka** for async order events, **API Gateway**, per-service databases (**PostgreSQL** for transactional services, **Cassandra** for high-write order history), **Redis** cache, and a **CI/CD pipeline** with **Canary** and **Blue-Green** deployments plus **centralized Logs/Metrics**. -e

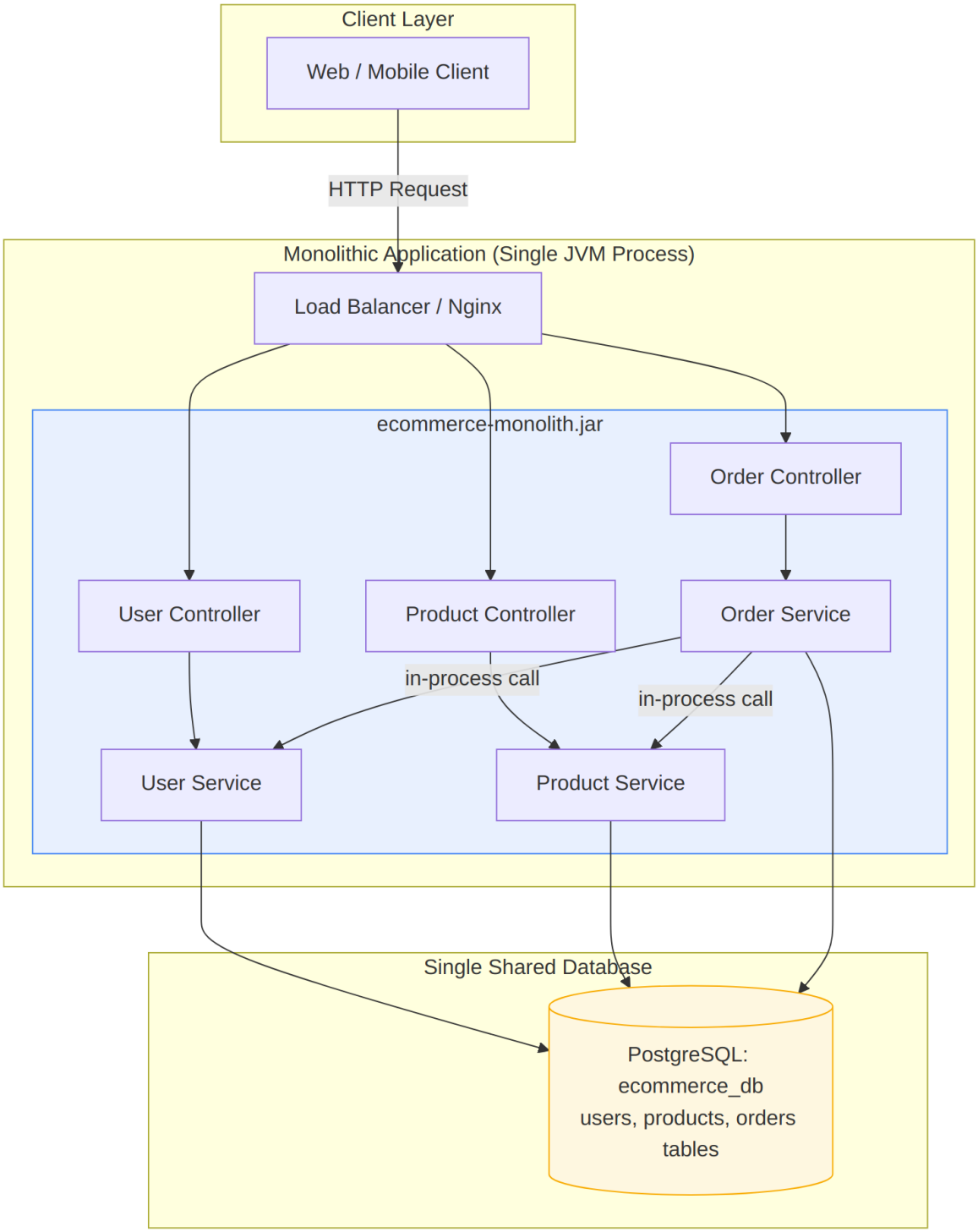


Figure 3: Architectural Flow Diagram

Part II — Modular Monolith → Microservices / Event-Driven → Serverless

---

Q1: Modular Monolith Architecture

Definition

A **Modular Monolith** is a single deployable application (like a classic monolith) but internally organized into **strictly-bounded modules**, each owning its own data and exposing a well-defined API to other modules. Modules **cannot** reach into each other's internal classes or tables directly — they communicate only through explicit interfaces or internal events.

Think of it as "microservices-shaped code, monolith-shaped deployment." It's the natural **stepping stone** between a tangled monolith and true microservices.

Aspect	Classic Monolith	Modular Monolith	Microservices
Deployment	1 unit	1 unit	N units
Module boundaries	Loose/none	Enforced at compile-time	Enforced by network/process
Communication	Any class calls any class	Interface/event only	REST/gRPC/Kafka
Database	1 shared schema	1 DB, schema-per-module	DB-per-service
Team ownership	Shared	Can be per-team	Per-team

Hands-On: Refactoring the E-Commerce Monolith into Modules (Spring Modulith)

We'll restructure our Part I monolith so `user`, `product`, and `order` become enforced modules using **Spring Modulith**, replacing direct service-to-service Java calls with **internal application events**.

```
<dependency>
    <groupId>org.springframework.modulith</groupId>
    <artifactId>spring-modulith-starter-core</artifactId>
    <version>1.2.0</version>
</dependency>
<dependency>
    <groupId>org.springframework.modulith</groupId>
    <artifactId>spring-modulith-starter-test</artifactId>
    <version>1.2.0</version>
    <scope>test</scope>
</dependency>
```

2.1 Add Spring Modulith dependency

2.2 Enforce module boundaries — package layout

```
com.example.ecommerce/
 ├── user/          (module: package-private classes, public API surface only)
 ├── product/
 ├── order/
 └── EcommerceApplication.java
```

Each package's internal classes (`UserRepository`, `UserEntity`) are marked **package-private**; only a thin public API (`UserApi` interface) is exposed.

2.3 Replace direct calls with domain events Before (tight coupling — Part I style):

```
userService.validateUser(userId);
productService.reserveStock(productId, qty);
```

After (event-driven inside the same JVM):

```
package com.example.ecommerce.order;

public record OrderPlacedEvent(Long orderId, Long userId, Long productId, Integer qty) {}
```

```
@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final ApplicationEventPublisher events;

    public OrderService(OrderRepository orderRepository, ApplicationEventPublisher events) {
        this.orderRepository = orderRepository;
        this.events = events;
    }

    public Order placeOrder(Long userId, Long productId, Integer qty) {
        Order order = new Order();
        order.setUserId(userId);
        order.setProductId(productId);
        order.setQuantity(qty);
        order.setStatus("CREATED");
        order = orderRepository.save(order);

        // Publish instead of directly calling ProductService/UserService
        events.publishEvent(new OrderPlacedEvent(order.getId(), userId, productId, qty));
        return order;
    }
}
```

```
package com.example.ecommerce.product;

@Component
public class ProductEventListener {

    private final ProductService productService;

    public ProductEventListener(ProductService productService) {
        this.productService = productService;
    }

    @ApplicationModuleListener   // Spring Modulith — transactional, async-capable
    void on(OrderPlacedEvent event) {
        productService.reserveStock(event.productId(), event.qty());
    }
}
```

```
class ModularityTests {

    ApplicationModules modules = ApplicationModules.of(EcommerceApplication.class);

    @Test
    void verifyModuleStructure() {
        modules.verify();   // Fails the build if 'order' reaches into 'product' internals directly
    }

    @Test
    void generateDocumentation() {
        new Documenter(modules).writeDocumentation();  // auto-generates module diagrams (PlantUML)
    }
}
```

2.4 Verify module boundaries automatically (test) Now `order` can no longer call `ProductRepository` directly — the build fails if someone bypasses the module API. This is the discipline that makes future extraction to microservices low-risk.

Real-Time Usage

- **Shopify, GitHub, Basecamp (early rewrites)** — all famously stayed on well-structured modular Rails/Ruby monoliths far longer than "microservices hype" suggested, because module discipline solved the coupling problem without paying the distributed-systems tax.
- **Mid-size SaaS companies** — teams that aren't yet at the scale/headcount to justify separate deployment pipelines per service, but still want team-level code ownership.
- **"Strangler Fig" migrations** — organizations use the modular monolith as an intermediate stage: once a module proves stable and has clear boundaries, it's the *cheapest possible* candidate to "cut out" into its own microservice, because the interface is already clean.

Architectural Flow Diagram

Q2: Microservices & Event-Driven Microservices

Definition

Microservices Architecture decomposes an application into small, **independently deployable services**, each: - Owns its **own database** (no shared schema) - Is developed, deployed, and scaled **independently** - Communicates over the network via **REST/gRPC** (synchronous) or a **message broker** (asynchronous)

Event-Driven Microservices extend this by making **events** (not direct API calls) the primary communication mechanism. Services publish facts ("OrderPlaced", "PaymentCompleted") to a broker like **Kafka**, and interested services subscribe — nobody calls anybody directly. This decouples services in *time* as well as *space*: the publisher doesn't need the subscriber to be online.

Sync Microservices (REST)	Event-Driven Microservices (Kafka)
Caller blocks waiting for response	Fire-and-forget / async
Tight temporal coupling (both must be up)	Loose coupling (broker buffers events)

Sync Microservices (REST)	Event-Driven Microservices (Kafka)
Simpler to reason about	Better resilience & scalability
Cascading failures possible	Natural backpressure via queue

Hands-On: Splitting the Monolith into 3 Microservices + Kafka

We now physically split `user`, `product`, `order` into **three separate Spring Boot applications**, each with **its own database**, communicating via **Kafka**.

3.1 Topology

```
user-service      → PostgreSQL (user_db)        : port 8081
product-service   → PostgreSQL (product_db)      : port 8082
order-service     → Cassandra (order_history_ks)  : port 8083
Kafka broker      → topics: order.placed, product.reserved, order.confirmed
```

(Order history is high-write, append-mostly, and queried mostly by order-id/user-id/time-range — a great fit for Cassandra's wide-column, write-optimized model, versus Postgres for User/Product's relational needs.)

```yaml
version: '3.8'
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.6.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181

  kafka:
    image: confluentinc/cp-kafka:7.6.0
    depends_on: [zookeeper]
    ports: ["9092:9092"]
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  postgres-user:
    image: postgres:16
    environment: { POSTGRES_DB: user_db, POSTGRES_PASSWORD: postgres }
    ports: ["5433:5432"]

  postgres-product:
    image: postgres:16
    environment: { POSTGRES_DB: product_db, POSTGRES_PASSWORD: postgres }
    ports: ["5434:5432"]

  cassandra:
    image: cassandra:5.0
    ports: ["9042:9042"]
```

3.2 `docker-compose.yml` (local infra)

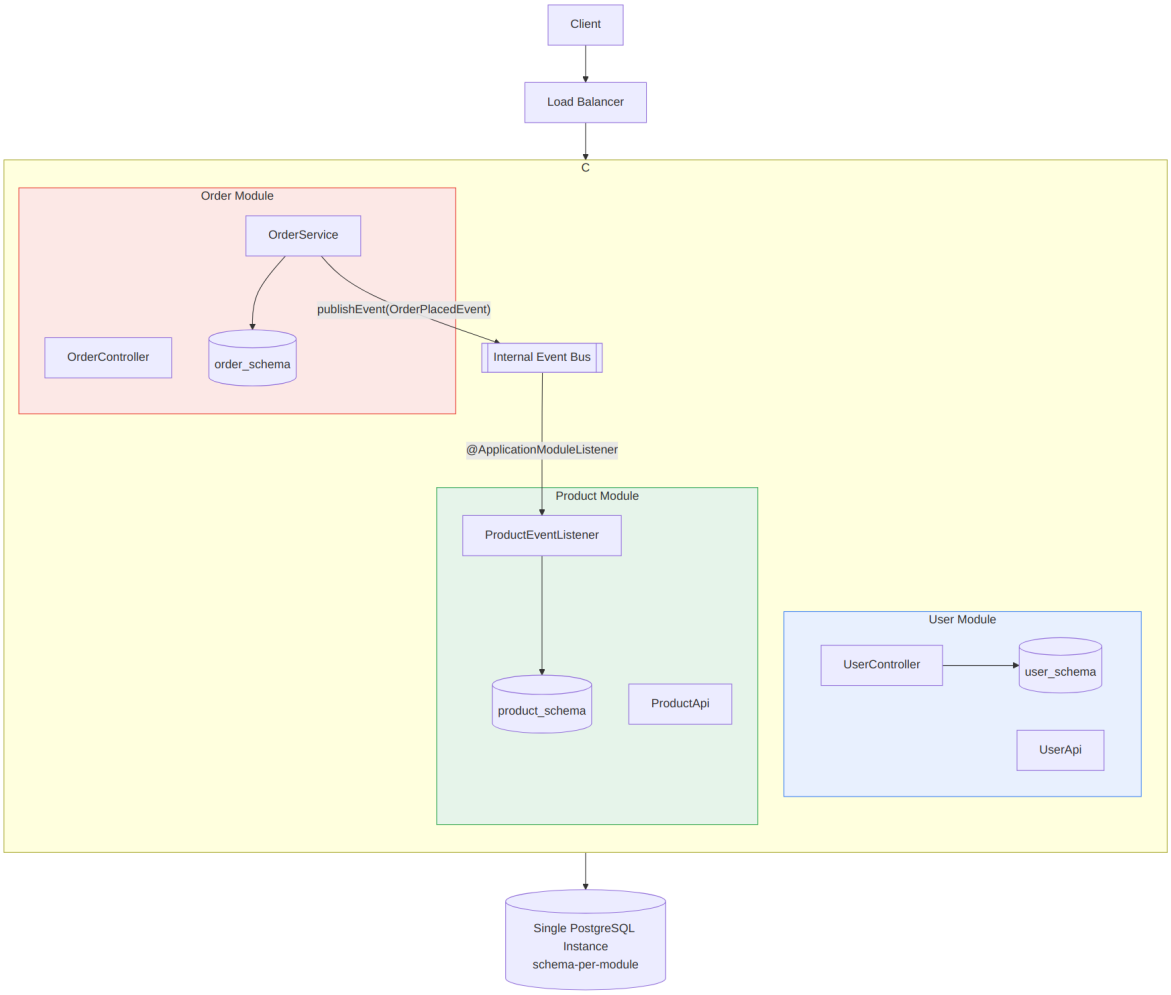


Figure 4: Architectural Flow Diagram

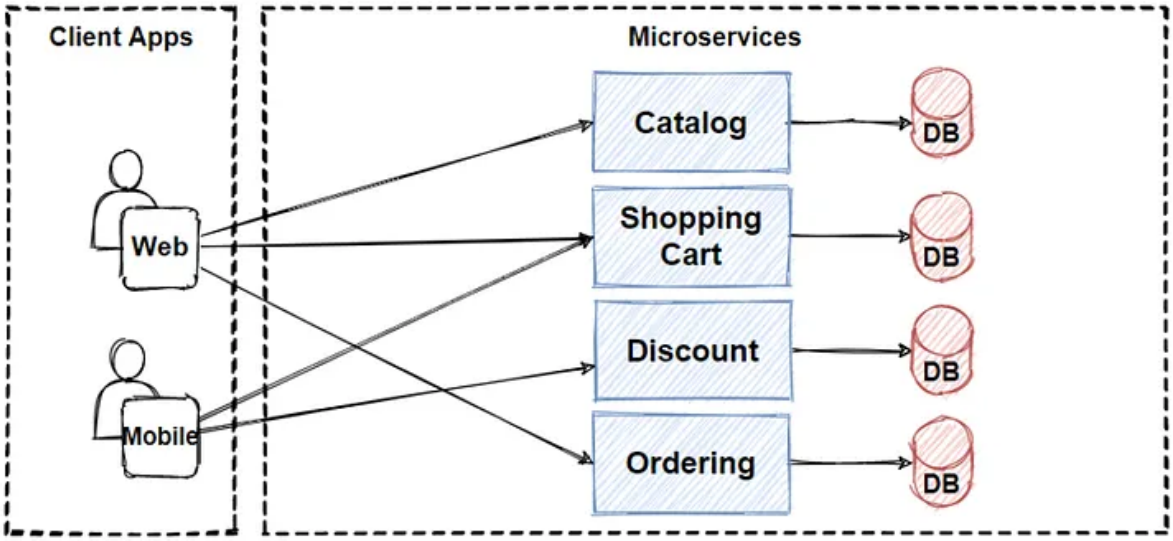


Figure 5: Basic Microservices Architecture — Client Apps calling independent services, each with its own DB

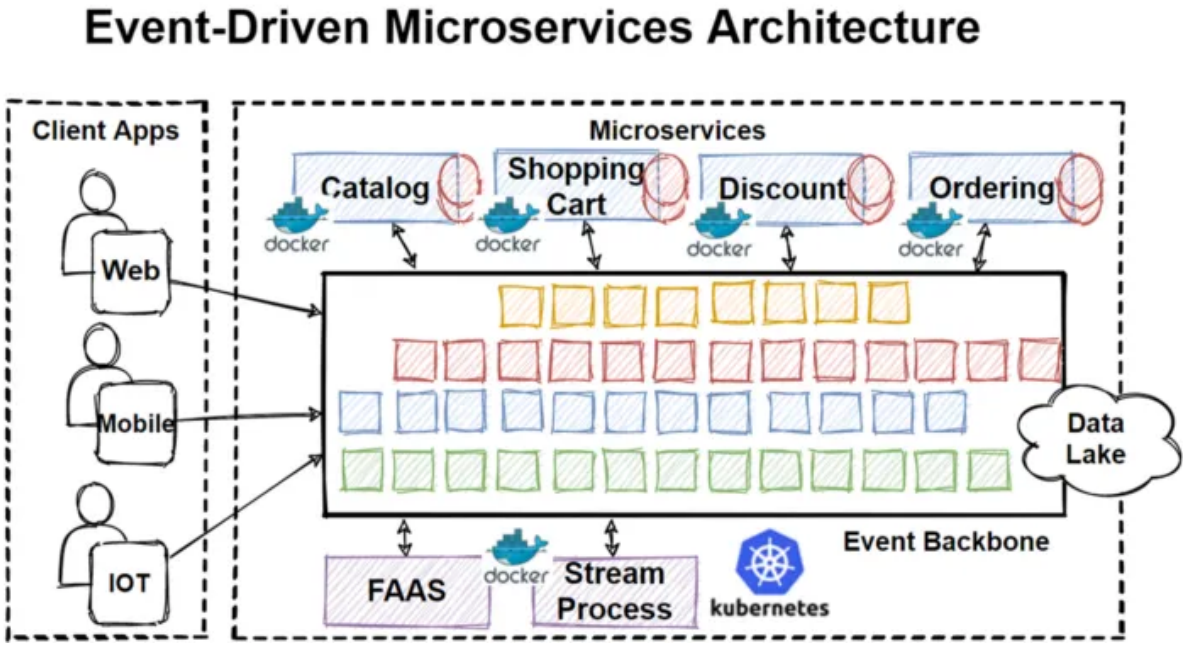


Figure 6: Event-Driven Microservices Architecture — Client Apps (Web/Mobile/IoT) publishing through an Event Backbone to Catalog, Shopping Cart, Discount, and Ordering microservices, with FaaS, Stream Processing, and a Data Lake consuming events

```java
package com.example.orderservice;

import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;
import org.springframework.web.bind.annotation.*;

@Service
class OrderProducer {
    private final KafkaTemplate<String, OrderPlacedEvent> kafkaTemplate;

    OrderProducer(KafkaTemplate<String, OrderPlacedEvent> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    void publish(OrderPlacedEvent event) {
        kafkaTemplate.send("order.placed", event.orderId().toString(), event);
    }
}

record OrderPlacedEvent(String orderId, Long userId, Long productId, Integer qty) {}

@RestController
@RequestMapping("/api/orders")
class OrderController {

    private final OrderRepository repository; // Cassandra repository
    private final OrderProducer producer;

    OrderController(OrderRepository repository, OrderProducer producer) {
        this.repository = repository;
        this.producer = producer;
    }

    @PostMapping
    public OrderEntity create(@RequestBody CreateOrderRequest req) {
        OrderEntity order = new OrderEntity(java.util.UUID.randomUUID().toString(),
                req.userId(), req.productId(), req.qty(), "PENDING");
        repository.save(order);
        producer.publish(new OrderPlacedEvent(order.getId(), req.userId(), req.productId(), req.qty()));
        return order;
    }
}
```

3.3 `order-service` — Publishing an Event

```java
@Table("orders_by_id")
public class OrderEntity {
    @PrimaryKey
    private String id;
    private Long userId;
    private Long productId;
    private Integer qty;
    private String status;
    // constructor, getters/setters
}
```

3.4 `order-service` — Cassandra Entity

```
package com.example.productservice;

import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;

@Service
class ProductEventConsumer {

    private final ProductRepository repository;      // Postgres/JPA
    private final KafkaTemplate<String, Object> kafkaTemplate;

    ProductEventConsumer(ProductRepository repository, KafkaTemplate<String, Object> kafkaTemplate) {
        this.repository = repository;
        this.kafkaTemplate = kafkaTemplate;
    }

    @KafkaListener(topics = "order.placed", groupId = "product-service")
    public void handle(OrderPlacedEvent event) {
        var product = repository.findById(event.productId()).orElseThrow();

        if (product.getStock() >= event.qty()) {
            product.setStock(product.getStock() - event.qty());
            repository.save(product);
            kafkaTemplate.send("product.reserved",
                new ProductReservedEvent(event.orderId(), true));
        } else {
            kafkaTemplate.send("product.reserved",
                new ProductReservedEvent(event.orderId(), false));
        }
    }
}

record OrderPlacedEvent(String orderId, Long userId, Long productId, Integer qty) {}
record ProductReservedEvent(String orderId, boolean success) {}
```

3.5 `product-service` — Consuming the Event & Reacting

```
@KafkaListener(topics = "product.reserved", groupId = "order-service")
public void onProductReserved(ProductReservedEvent event) {
    OrderEntity order = repository.findById(event.orderId()).orElseThrow();
    order.setStatus(event.success() ? "CONFIRMED" : "FAILED");
    repository.save(order);
    // publish order.confirmed / order.failed for downstream (notifications, billing) ...
}
```

3.6 `order-service` — Completing the Saga This is a simple **Choreography-based Saga**: no central orchestrator — each service reacts to events and emits the next event. (An alternative is **Orchestration**, where a saga-orchestrator service explicitly calls each step and manages compensation.)

```
cd user-service    && mvn spring-boot:run &
cd product-service && mvn spring-boot:run &
cd order-service   && mvn spring-boot:run &

# Scale ONLY the hot path independently — e.g. 5x order-service during a flash sale
docker service scale order-service=5
```

3.7 Run each service independently

Real-Time Usage

- **E-commerce checkout** (Amazon, Flipkart): order, inventory, payment, and shipping are separate services; Kafka topics like `order.placed` fan out to inventory, payment, notification, and analytics services simultaneously.
- **Uber/Lyft ride matching**: `trip-service`, `driver-location-service`, `pricing-service`, and `matching-service` communicate via event streams so location updates (very high volume) don't block trip booking.
- **Banking/Fintech**: `payment-service` emits `PaymentCompleted`; `ledger-service`, `fraud-detection-service`, and `notification-service` consume independently — critical for audit trails and eventual consistency across bounded contexts.
- **Netflix**: hundreds of microservices, heavy use of async messaging for recommendations, billing, and playback telemetry, decoupling teams that ship independently many times a day.

Architectural Flow Diagram

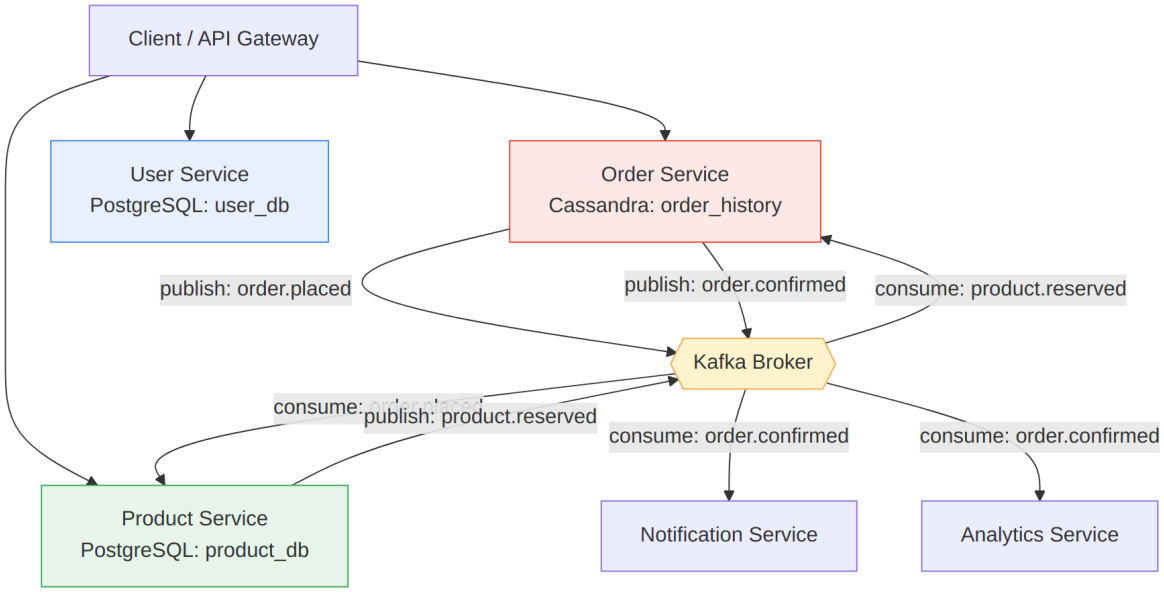


Figure 7: Architectural Flow Diagram

Q3: Serverless Architecture — Designing for Millions of Requests

Definition

Serverless Architecture means running code as **event-triggered functions** (FaaS — Function as a Service, e.g., AWS Lambda, Azure Functions, Google Cloud Functions) without provisioning or managing servers. The cloud provider handles scaling, patching, and availability; you pay only for actual execution time.

To handle **millions of requests reliably**, a serverless design combines several building blocks:

Layer	Purpose
CDN (CloudFront/Fastly)	Cache static/API responses at the edge, absorb read traffic
API Gateway	Entry point, request throttling, auth, request validation
Lambda (compute)	Stateless business logic, auto-scales per-request
SQS / Kinesis / EventBridge	Buffer & decouple bursts from downstream processing (backpressure)
DynamoDB / Aurora Serverless	Auto-scaling data layer, single-digit ms reads
ElastiCache (Redis)	Hot-path caching to cut DB load
Step Functions	Orchestrate multi-step workflows without a "server"
Provisioned Concurrency	Pre-warm Lambdas to eliminate cold-start latency for critical paths

Hands-On: Serverless Order-Processing Pipeline (Java + AWS)

Goal: absorb a flash-sale spike of order requests without dropping any, using a **buffer-and-drain** pattern.

4.1 Flow

```
Client → API Gateway → Lambda (validate + enqueue) → SQS → Lambda (process order) → DynamoDB
                                                                        ↓
                                                                  SNS → Notification
```

```java
package com.example.serverless;

import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.APIGatewayProxyRequestEvent;
import com.amazonaws.services.lambda.runtime.events.APIGatewayProxyResponseEvent;
import com.amazonaws.services.sqs.AmazonSQS;
import com.amazonaws.services.sqs.AmazonSQSClientBuilder;
import com.amazonaws.services.sqs.model.SendMessageRequest;
import com.fasterxml.jackson.databind.ObjectMapper;

public class ValidateOrderHandler
        implements RequestHandler<APIGatewayProxyRequestEvent, APIGatewayProxyResponseEvent> {

    private final AmazonSQS sqs = AmazonSQSClientBuilder.defaultClient();
```

```
    private final ObjectMapper mapper = new ObjectMapper();
    private static final String QUEUE_URL = System.getenv("ORDER_QUEUE_URL");

    @Override
    public APIGatewayProxyResponseEvent handleRequest(APIGatewayProxyRequestEvent request, Context context) {
        try {
            OrderRequest order = mapper.readValue(request.getBody(), OrderRequest.class);

            if (order.getQty() <= 0) {
                return new APIGatewayProxyResponseEvent().withStatusCode(400)
                        .withBody("{\"error\":\"invalid quantity\"}");
            }

            // Enqueue instead of processing synchronously — absorbs bursts
            sqs.sendMessage(new SendMessageRequest()
                    .withQueueUrl(QUEUE_URL)
                    .withMessageBody(mapper.writeValueAsString(order)));

            return new APIGatewayProxyResponseEvent().withStatusCode(202)
                    .withBody("{\"status\":\"accepted\"}");

        } catch (Exception e) {
            return new APIGatewayProxyResponseEvent().withStatusCode(500)
                    .withBody("{\"error\":\"" + e.getMessage() + "\"}");
        }
    }
}
```

4.2 API Gateway → Validation Lambda (Java)

```
package com.example.serverless;

import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.SQSEvent;
import com.amazonaws.services.dynamodbv2.AmazonDynamoDB;
import com.amazonaws.services.dynamodbv2.AmazonDynamoDBClientBuilder;
import com.amazonaws.services.dynamodbv2.model.*;
import com.fasterxml.jackson.databind.ObjectMapper;

import java.util.HashMap;
import java.util.Map;

public class ProcessOrderHandler implements RequestHandler<SQSEvent, Void> {

    private final AmazonDynamoDB dynamoDB = AmazonDynamoDBClientBuilder.defaultClient();
    private final ObjectMapper mapper = new ObjectMapper();

    @Override
    public Void handleRequest(SQSEvent event, Context context) {
        for (SQSEvent.SQSMessage message : event.getRecords()) {
            try {
                OrderRequest order = mapper.readValue(message.getBody(), OrderRequest.class);

                Map<String, AttributeValue> item = new HashMap<>();
                item.put("orderId", new AttributeValue(java.util.UUID.randomUUID().toString()));
                item.put("userId", new AttributeValue(order.getUserId().toString()));
```

```
                item.put("productId", new AttributeValue(order.getProductId().toString()));
                item.put("status", new AttributeValue("CONFIRMED"));

                dynamoDB.putItem(new PutItemRequest("Orders", item));

            } catch (Exception e) {
                context.getLogger().log("Failed to process message: " + e.getMessage());
                throw new RuntimeException(e); // triggers SQS retry / DLQ after max receives
            }
        }
        return null;
    }
}
```

4.3 SQS → Processing Lambda (Java) — writes to DynamoDB

```
# serverless.yml (Serverless Framework) — excerpt
functions:
  validateOrder:
    handler: com.example.serverless.ValidateOrderHandler
    reservedConcurrency: 500      # cap to protect downstream
    events:
      - httpApi: 'POST /orders'

  processOrder:
    handler: com.example.serverless.ProcessOrderHandler
    provisionedConcurrency: 20    # keep warm to avoid cold starts under load
    events:
      - sqs:
          arn: arn:aws:sqs:us-east-1:xxxx:order-queue
          batchSize: 10
          maximumBatchingWindow: 2

resources:
  Resources:
    OrderQueue:
      Type: AWS::SQS::Queue
      Properties:
        RedrivePolicy:
          deadLetterTargetArn: !GetAtt OrderDLQ.Arn
          maxReceiveCount: 3
    OrdersTable:
      Type: AWS::DynamoDB::Table
      Properties:
        BillingMode: PAY_PER_REQUEST   # auto-scales with traffic, no capacity planning
```

4.4 Infra-as-code sketch (key scaling knobs) Why this survives millions of requests:
- **API Gateway + Lambda** scale horizontally per-request automatically (no capacity planning).
- **SQS** absorbs bursts — if 1M requests arrive in 1 minute, they queue safely instead of overwhelming downstream systems; `processOrder` Lambdas drain the queue at a controlled, scalable rate. - **DynamoDB on-demand** scales partitions automatically for high write throughput. - **Dead-letter queue (DLQ)** captures poison messages after retries, preventing one bad message from blocking the pipeline. - **Provisioned concurrency** removes cold-start latency for the critical processing path.

Real-Time Usage

- **Flash sales / Black Friday** (retail): traffic can spike 50–100x in minutes — serverless auto-scales without pre-provisioning servers for peak capacity you'd otherwise pay for year-round.
- **IoT telemetry ingestion**: millions of devices sending small events — API Gateway/IoT Core + Lambda + Kinesis handle massive fan-in cheaply, paying only per invocation.
- **Image/video processing pipelines**: S3 upload triggers a Lambda to generate thumbnails/transcodes — naturally event-driven, bursty, and embarrassingly parallel.
- **Chatbots & notification fan-out**: Slack/WhatsApp webhook → Lambda → SNS fan-out to millions of subscribers without maintaining a fleet of servers.

Architectural Flow Diagram

Coming Next — Part III

CI/CD Pipeline Deep Dive: automated build/test/deploy pipelines for the microservices above, plus **Canary Releases** and **Blue-Green Deployments** for zero-downtime rollout, and **centralized Logs (ELK/Loki) & Metrics (Prometheus/Grafana)** for observability across the whole system. -e

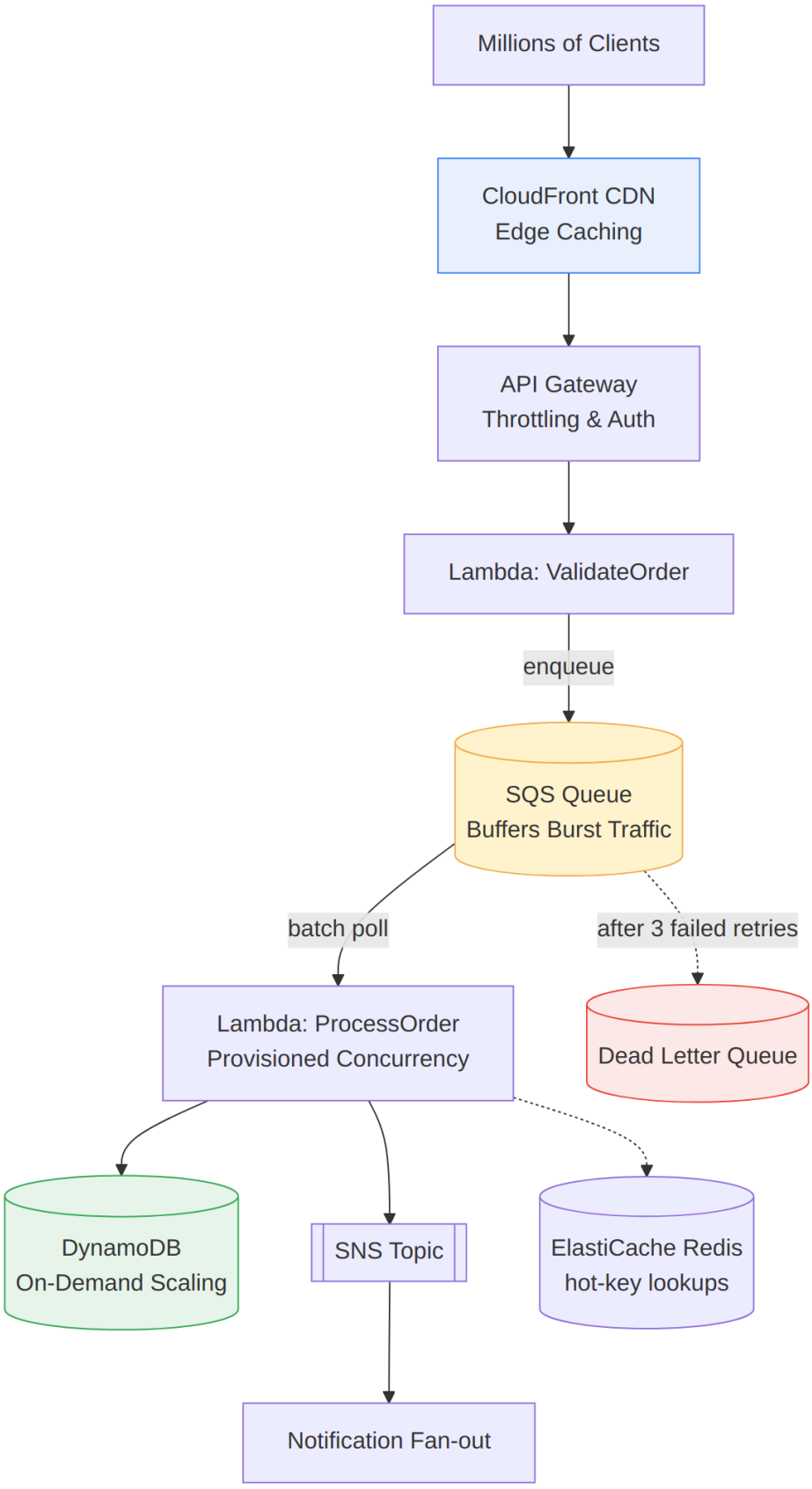


Figure 8: Architectural Flow Diagram

Part III — Decomposition • Communication • Data Management • Caching • Deployment

---

Q1: Microservices Decomposition

Definition

Decomposition is the discipline of deciding *where the boundaries go* when splitting a system into services. Getting this wrong is the #1 cause of "distributed monoliths" (many services, but still tightly coupled). Common strategies:

Strategy	Approach
Decompose by Business Capability	One service per business function (Order Management, Inventory, Payments, Shipping) — mirrors org chart / Conway's Law
Decompose by Subdomain (DDD)	Use Domain-Driven Design to find **Bounded Contexts** — each has its own ubiquitous language and model (e.g., "Product" means something different to Catalog vs Warehouse)
Strangler Fig Pattern	Incrementally carve functionality out of a monolith behind a facade/gateway, redirecting traffic route-by-route until the monolith is "strangled" to nothing
Decompose by Transaction/Use Case	Group code that changes together and is invoked together in one use case, to minimize cross-service chatty calls

Hands-On: Applying DDD + Strangler Fig to the E-Commerce Monolith

1.1 Bounded Context Mapping

```
Monolith Domain Model → Bounded Contexts:

┌─────────────┬──────────────────────────────────────────┐
│ Context     │ Owns                                     │
├─────────────┼──────────────────────────────────────────┤
│ Catalog     │ Product info, pricing, categories        │
│ Inventory   │ Stock levels, warehouse locations        │
│ Ordering    │ Cart, order lifecycle, order history     │
│ Payments    │ Payment methods, transactions, refunds   │
│ Shipping    │ Fulfillment, tracking, carrier integration │
│ Identity    │ Users, auth, profile                     │
└─────────────┴──────────────────────────────────────────┘
```

Note: "Product" appears in *both* Catalog (rich description, price) and Inventory (SKU, stock count only) — each context keeps its **own slim model**, not one shared "Product" entity. This avoids the classic mistake of a god-class `Product` table shared by five services.

1.2 Strangler Fig via API Gateway routing (Spring Cloud Gateway) We extract `Order` first, while `User` and `Product` still live in the monolith.

```yaml
# gateway application.yml
spring:
  cloud:
    gateway:
      routes:
        - id: order-service-new
          uri: http://order-service:8083
          predicates:
            - Path=/api/orders/**
          # NEW traffic goes to the extracted microservice

        - id: legacy-monolith
          uri: http://ecommerce-monolith:8080
          predicates:
            - Path=/api/users/**, /api/products/**
          # Everything else still hits the old monolith
```

```java
// Feature-flag based cutover for a gradual migration (percentage-based routing)
@Bean
public RouteLocator strangerFigRoutes(RouteLocatorBuilder builder) {
    return builder.routes()
        .route("order-canary", r -> r.path("/api/orders/**")
            .and().weight("order-group", 20)   // 20% to new microservice
            .uri("http://order-service:8083"))
        .route("order-legacy", r -> r.path("/api/orders/**")
            .and().weight("order-group", 80)   // 80% still to monolith
            .uri("http://ecommerce-monolith:8080"))
        .build();
}
```

Once confidence is high (error rates, latency, and data parity all validated), the weight shifts to 100% on `order-service`, and the `Order` code + tables are deleted from the monolith.

Real-Time Usage

- **Amazon (2001-2002 SOA migration)** — the original "two-pizza team" mandate forced decomposition by business capability so teams could ship independently.
- **Netflix** — decomposed by subdomain (membership, viewing history, recommendations, billing), enabling hundreds of independent deploys per day.
- **Legacy bank core migrations** — banks routinely use Strangler Fig over multi-year programs because a "big bang" rewrite of a core ledger is too risky; they carve out one bounded context (e.g., "Card Payments") at a time behind a gateway.

Architectural Flow Diagram

Q2: Microservices Communication (Sync / Async, API Gateway, Rest-Template, Feign, gRPC)

Definition

Client and services communicate with each other through many different mechanisms, but they mainly split along one axis: **Synchronous vs Asynchronous**.

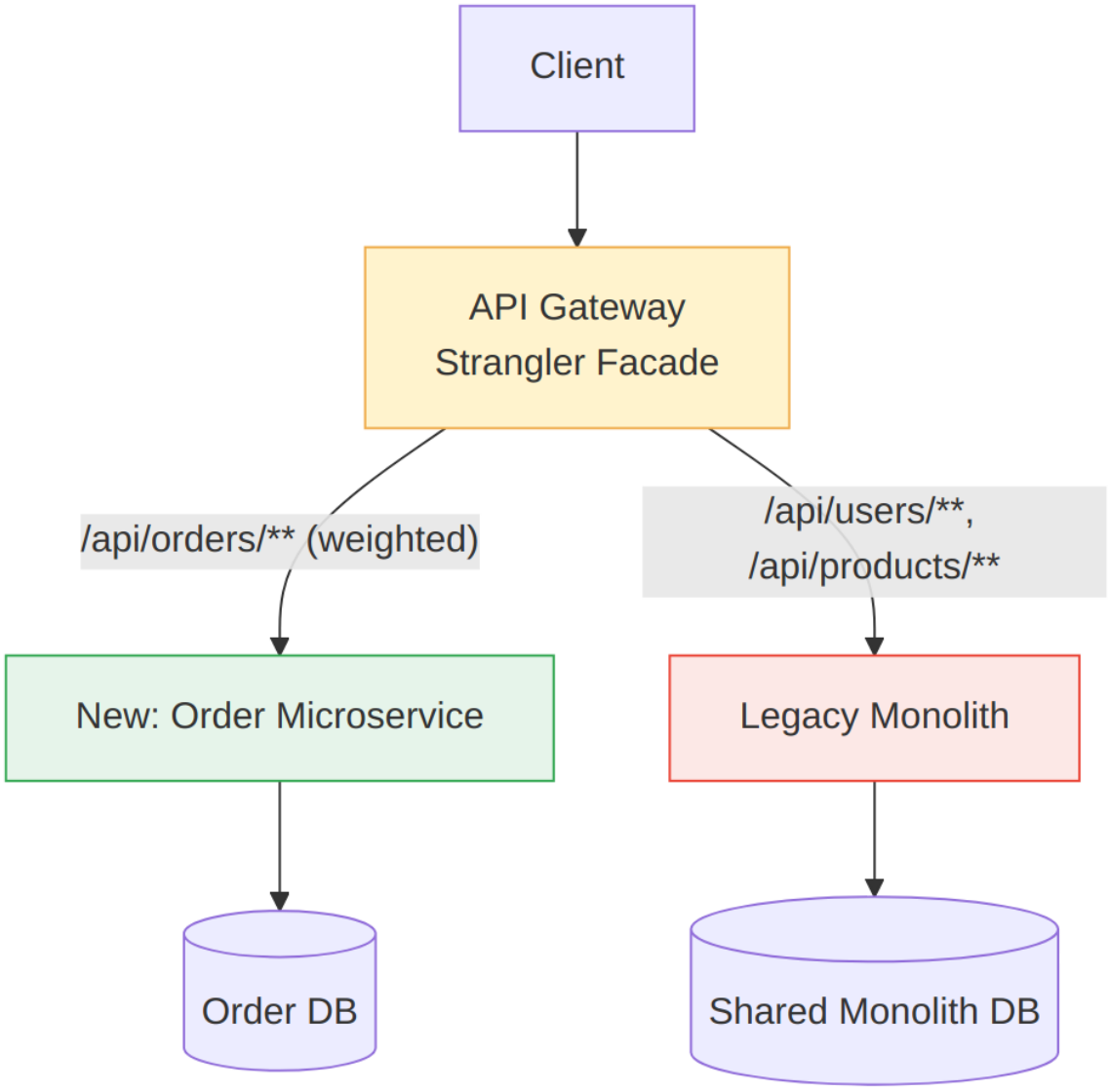


Figure 9: Architectural Flow Diagram

Synchronous communication: the client sends a request and **waits** for a response from the service — the client's thread is **blocked** until the response arrives. The client's code only continues once the HTTP (or gRPC) response comes back. This is simple to reason about, but it couples the *availability* of the caller and callee: if the callee is slow or down, the caller is stuck waiting too.

Asynchronous communication: the client sends a request (usually a message) but does **not** wait for a response — its thread is free to continue immediately. The most common protocol for this is **AMQP (Advanced Message Queuing Protocol)**: the client (producer) publishes a message to a broker like **Kafka** or **RabbitMQ**, and one or more consumers pick it up independently, with nobody blocking on a reply.

Asynchronous systems further split by delivery model: - **One-to-one (queue)**: a single producer, a single receiver — the classic work-queue. - **One-to-many (topic)**: multiple receivers; each message can be processed by zero-to-many subscribers. Topic-based delivery is inherently asynchronous — this is the publish/subscribe mechanism behind Event-Driven Microservices (Part II).

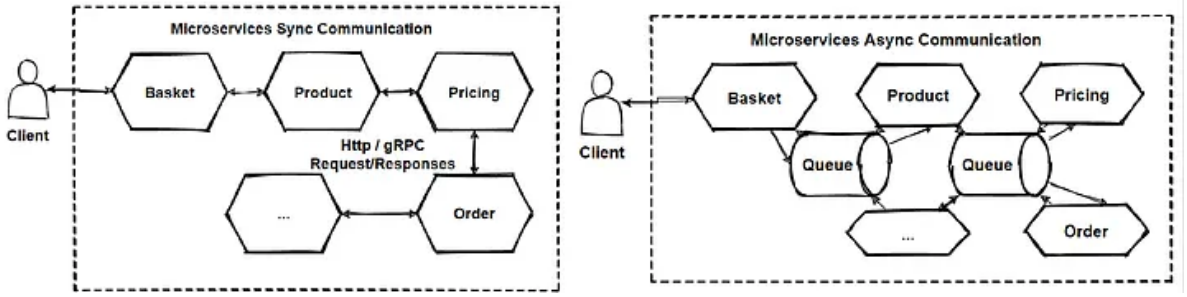


Figure 10: Microservices Sync vs Async Communication — Sync: Basket, Product, and Pricing calling each other directly via HTTP/gRPC request-response; Async: the same services communicating through Queues, decoupling producers and consumers

- **API Gateway**: a single entry point that handles routing, auth, rate-limiting, and aggregation, so clients don't need to know about internal service topology.

Client	Style	Use case
RestTemplate	Sync, blocking, imperative HTTP client (legacy, being phased out in favor of `WebClient`)	Simple service-to-service REST calls
Feign Client	Sync, declarative HTTP client — write an interface, Spring generates the implementation	Cleaner code, built-in load-balancing with Eureka/Ribbon
gRPC	Sync (or streaming), binary Protobuf over HTTP/2	Low-latency, high-throughput internal calls (e.g., between `order-service` and `inventory-service`)
Kafka/RabbitMQ	Async	Event-driven flows (as built in Part II)

Designing HTTP-Based RESTful APIs for Microservices

When using synchronous request/response communication, **REST over HTTP** is the most common way to design APIs — especially for anything exposed **outside** the microservice cluster. For communication **internal** to the cluster, a binary protocol like **gRPC** is often the better choice, since it's built for speed rather than human-readable payloads.

Good API design matters a lot in microservices: since data only moves through messages/API calls, APIs must be **efficient (not chatty)**, well-documented, and **versioned** so updates don't break other services. There are two categories of synchronous API to design for:

1. **Public APIs** — calls made *from client applications* (web, mobile). These should stay **RESTful over HTTP with JSON payloads**: JSON is easy to read, easy to debug, and easy to agree on as a contract with client teams.
2. **Backend APIs** — calls made *between* backend microservices. Here, **network performance** matters more than human-readable payloads, since inter-service traffic can be very chatty. This is where **binary protocols like gRPC** become the better choice: smaller payloads, faster serialization.

	REST	gRPC
Transport	HTTP, JSON	HTTP/2, binary Protobuf
Interface	HTTP verbs (GET/POST/PUT/DELETE)	Remote Procedure Call — invoke a method on a remote system directly
Readability	Human-readable payloads	Not human-readable, but much faster
Best for	Public/client-facing APIs	Internal service-to-service calls

Even though both a client's REST call and an internal gRPC call are *synchronous*, we deliberately pick different protocols for each: the client-facing call favors visibility into the payload, while the backend call sacrifices readability for raw speed.

Hands-On

```
@Configuration
public class RestClientConfig {
    @Bean
    public RestTemplate restTemplate(RestTemplateBuilder builder) {
        return builder
            .setConnectTimeout(Duration.ofSeconds(2))
            .setReadTimeout(Duration.ofSeconds(3))
            .build();
    }
}

@Service
public class ProductLookupService {
    private final RestTemplate restTemplate;

    public ProductLookupService(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }
```

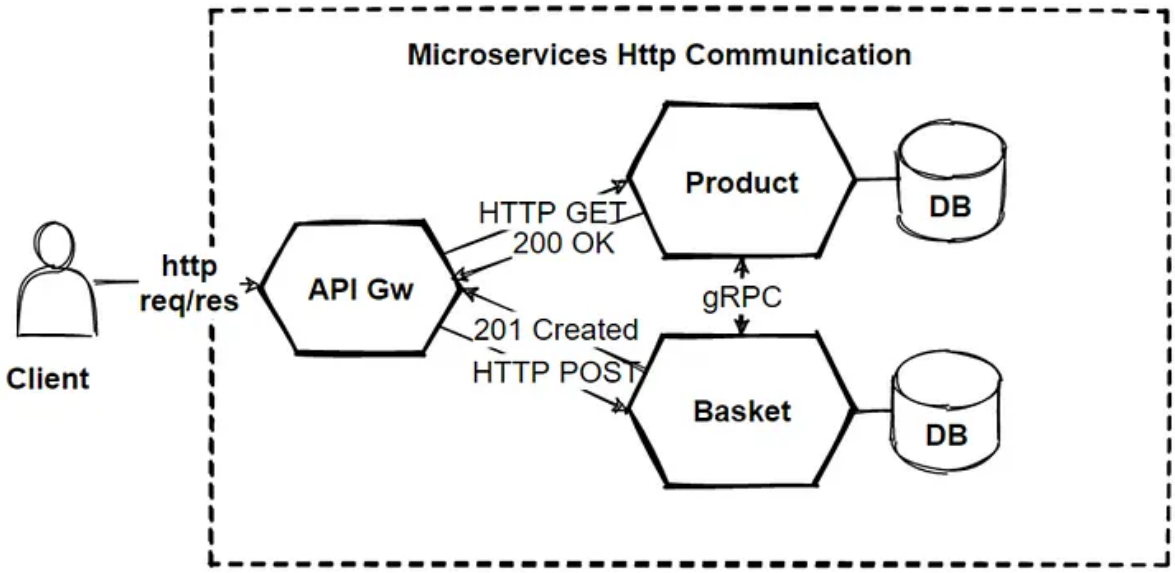


Figure 11: Microservices HTTP Communication with an API Gateway — Client sends http req/res to the API Gateway, which does HTTP GET (200 OK) to Product and HTTP POST (201 Created) to Basket, while Product and Basket talk to each other internally over gRPC

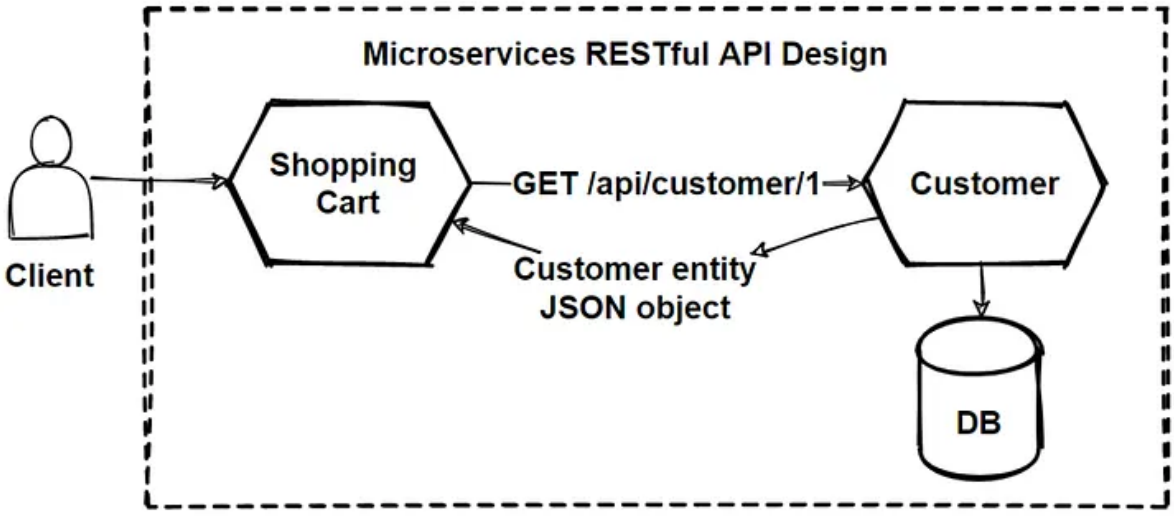


Figure 12: Microservices RESTful API Design — Client calls Shopping Cart, which issues GET /api/customer/1 to the Customer service, which queries its DB and returns a Customer entity as a JSON object

```
    public ProductDto getProduct(Long id) {
        return restTemplate.getForObject(
            "http://product-service/api/products/{id}", ProductDto.class, id);
    }
}
```

2.1 RestTemplate (synchronous, imperative)

```
@EnableFeignClients
@SpringBootApplication
public class OrderServiceApplication { }

@FeignClient(name = "product-service", fallback = ProductClientFallback.class)
public interface ProductClient {
    @GetMapping("/api/products/{id}")
    ProductDto getProduct(@PathVariable("id") Long id);

    @PutMapping("/api/products/{id}/reserve")
    void reserveStock(@PathVariable("id") Long id, @RequestParam int qty);
}

@Component
class ProductClientFallback implements ProductClient {
    public ProductDto getProduct(Long id) {
        return ProductDto.unavailable(id);   // Circuit breaker fallback
    }
    public void reserveStock(Long id, int qty) {
        throw new ServiceUnavailableException("product-service down, order held for retry");
    }
}
```

```
// Usage — reads like a local method call, but it's a real HTTP request under the hood
@Service
public class OrderService {
    private final ProductClient productClient;

    public Order placeOrder(Long productId, int qty) {
        ProductDto product = productClient.getProduct(productId);
        productClient.reserveStock(productId, qty);
        // ...
    }
}
```

2.2 Feign Client (declarative, preferred in Spring Cloud stacks)

```
// inventory.proto
syntax = "proto3";
option java_package = "com.example.inventory.grpc";

service InventoryService {
  rpc ReserveStock (ReserveRequest) returns (ReserveResponse);
}

message ReserveRequest {
```

```
  int64 productId = 1;
  int32 quantity = 2;
}

message ReserveResponse {
  bool success = 1;
  string message = 2;
}
```

```
// Server-side implementation (inventory-service)
@GrpcService
public class InventoryGrpcService extends InventoryServiceGrpc.InventoryServiceImplBase {
    @Override
    public void reserveStock(ReserveRequest req, StreamObserver<ReserveResponse> responseObserver) {
        boolean ok = inventoryRepository.tryReserve(req.getProductId(), req.getQuantity());
        ReserveResponse response = ReserveResponse.newBuilder()
                .setSuccess(ok)
                .setMessage(ok ? "Reserved" : "Insufficient stock")
                .build();
        responseObserver.onNext(response);
        responseObserver.onCompleted();
    }
}
```

```
// Client-side (order-service) — auto-injected stub via grpc-spring-boot-starter
@Service
public class OrderService {
    @GrpcClient("inventory-service")
    private InventoryServiceGrpc.InventoryServiceBlockingStub inventoryStub;

    public void reserve(Long productId, int qty) {
        ReserveResponse resp = inventoryStub.reserveStock(
            ReserveRequest.newBuilder().setProductId(productId).setQuantity(qty).build());
        if (!resp.getSuccess()) throw new InsufficientStockException(resp.getMessage());
    }
}
```

2.3 gRPC (high-performance internal call)

```
@RestController
public class OrderDetailsAggregatorController {
    private final ProductClient productClient;   // Feign
    private final RestTemplate userClient;        // RestTemplate

    @GetMapping("/api/order-details/{orderId}")
    public OrderDetailsResponse aggregate(@PathVariable String orderId) {
        Order order = orderRepository.findById(orderId);
        ProductDto product = productClient.getProduct(order.getProductId());
        UserDto user = userClient.getForObject("http://user-service/api/users/{id}",
                UserDto.class, order.getUserId());
        return new OrderDetailsResponse(order, product, user);
    }
}
```

2.4 API Gateway aggregating multiple backend calls

Real-Time Usage

- **Feign + Circuit Breaker (Resilience4j)**: standard in most Spring Cloud shops (retail, banking) to call internal REST services with automatic fallback if a downstream service is degraded.
- **gRPC**: used heavily at Google, Netflix, and in latency-sensitive internal hops (e.g., a `pricing-service` called millions of times/sec internally by `search-service`) where JSON/HTTP overhead is too costly.
- **API Gateway aggregation (BFF pattern)**: mobile apps often call one Gateway endpoint that fans out to 3-4 backend services and merges the result, minimizing round trips over cellular networks.

Q3: Microservices Data Management (Databases, CQRS, Event Sourcing, Distributed Transactions)

Definition

Each microservice owns its **own database** — no shared schema, no cross-service SQL joins. This creates two hard problems:

1. **Distributed transactions**: How do you keep data consistent across services without a single ACID transaction? → Solved with **Sagas** (seen in Part II) instead of 2-Phase-Commit.
2. **Cross-service queries**: How do you show "Order + Product + User" together if each lives in a different DB? → Solved with **CQRS** and **materialized read views**.

CQRS (Command Query Responsibility Segregation) is one of the critical patterns when querying across microservices. It separates **read** and **update** operations for a database, letting us avoid complex cross-service queries and inefficient joins.

To isolate Commands and Queries, best practice is to split the read and write sides into **two databases**. If an application is read-intensive (reads far more than it writes), the read side can use a custom data schema **optimized purely for queries**. The **Materialized View pattern** is the classic way to implement this: pre-defined, fine-grained, denormalized data avoids complex joins and mappings at query time.

Because the two sides are isolated, they don't even need to be the same *kind* of database — a common combination is a **NoSQL document database for reads** (fast, denormalized, horizontally scalable) paired with a **relational database for writes** (strong consistency for CRUD/transactional operations).

Event Sourcing is the pattern most commonly used *together with* CQRS. Instead of storing only the *current state* of a record, Event Sourcing stores the full, ordered sequence of **events** that led to that state (`OrderCreated`, `ItemAdded`, `ItemDeleted`, `OrderShipped`...).

When CQRS is combined with Event Sourcing, the core idea is: the **write database becomes an event store** — the source of truth — holding every event in sequential order rather than the latest status. Instead of updating a row in place, each change is **appended** as a new event. The **read side** of CQRS then provides materialized views built from **denormalized tables**, which consume events from the write side (event store) and convert them into query-optimized read models. This event-to-read-model conversion is typically handled by a **publish/subscribe pattern** via a message broker, sitting between the event store and the read database.

This gives a perfect audit log (nothing is ever overwritten or lost) and lets you rebuild any read-model from scratch at any time by **replaying** the full event history.

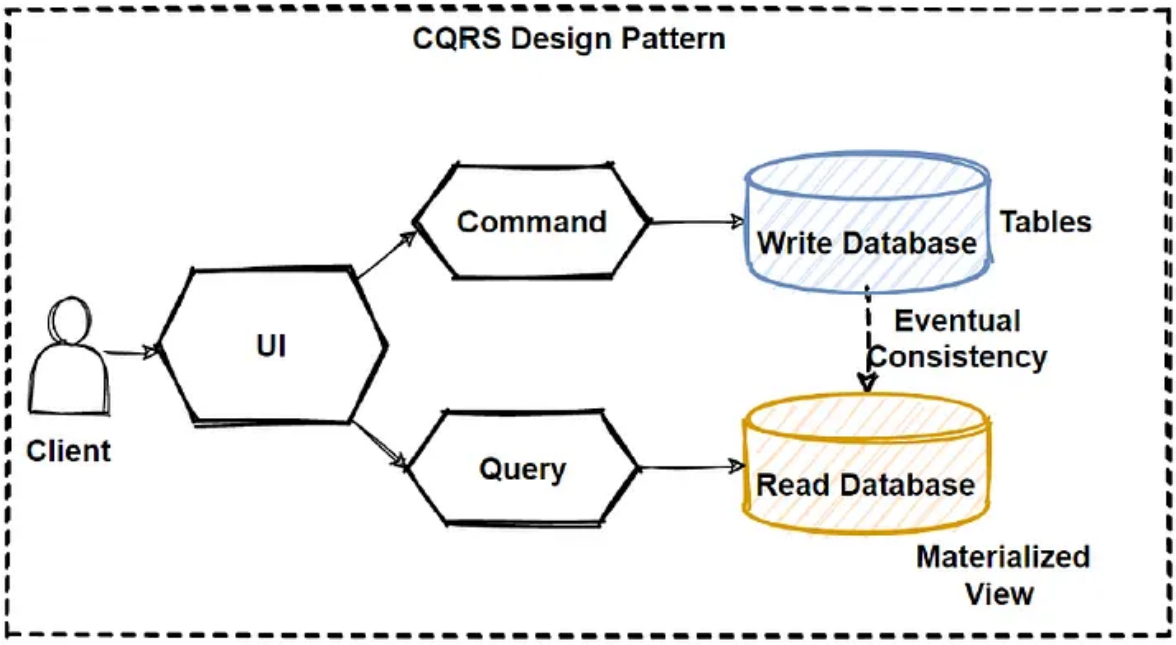


Figure 13: CQRS Design Pattern — UI splits into Command (writes to Write Database tables) and Query (reads from a Read Database materialized view), connected via eventual consistency

Hands-On: CQRS + Event Sourcing for Orders (PostgreSQL write side, Cassandra read side)

```
CREATE TABLE order_events (
    id BIGSERIAL PRIMARY KEY,
    aggregate_id VARCHAR(64) NOT NULL,    -- order id
    event_type VARCHAR(64) NOT NULL,      -- OrderCreated, ItemAdded, OrderShipped
    payload JSONB NOT NULL,
    version INT NOT NULL,
    created_at TIMESTAMP DEFAULT now()
);
```

```
@Service
public class OrderCommandService {

    private final OrderEventRepository eventRepository;  // JPA -> Postgres
    private final KafkaTemplate<String, DomainEvent> kafkaTemplate;

    public void createOrder(CreateOrderCommand cmd) {
        OrderCreatedEvent event = new OrderCreatedEvent(cmd.orderId(), cmd.userId(), cmd.productId(),
↪   cmd.qty());

        eventRepository.save(new OrderEventEntity(
                cmd.orderId(), "OrderCreated", toJson(event), 1));

        // Publish so read-side projections can update
        kafkaTemplate.send("order.events", cmd.orderId(), event);
    }
```

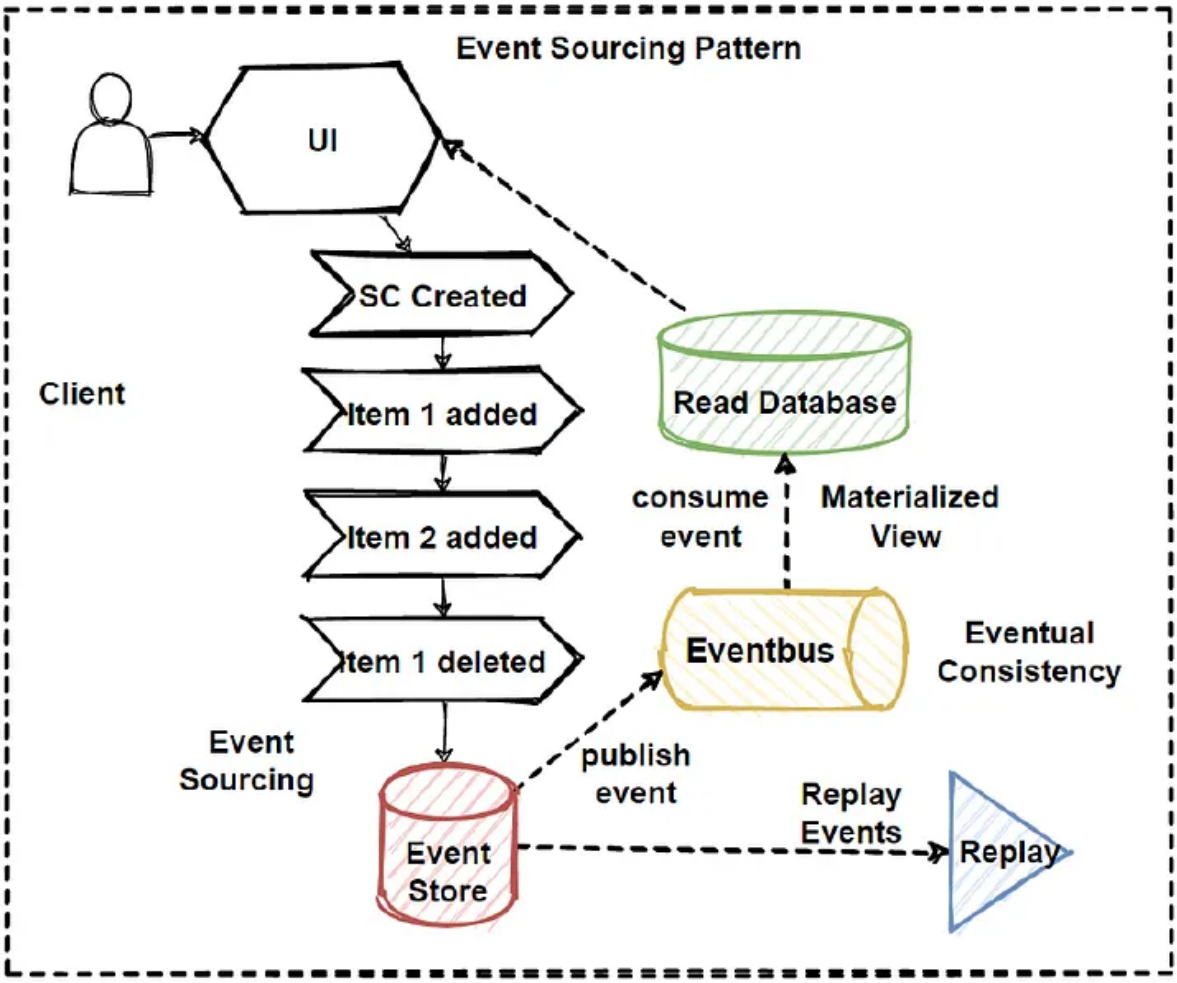


Figure 14: Event Sourcing Pattern — UI actions (SC Created, Item Added, Item Deleted) append to an Event Store, publish to an Eventbus, get consumed into a Read Database materialized view, and can be Replayed

```
    public void shipOrder(String orderId) {
        int nextVersion = eventRepository.findMaxVersion(orderId) + 1;
        OrderShippedEvent event = new OrderShippedEvent(orderId);
        eventRepository.save(new OrderEventEntity(orderId, "OrderShipped", toJson(event), nextVersion));
        kafkaTemplate.send("order.events", orderId, event);
    }
}
```

3.1 Write side — Event Store (PostgreSQL)

```
public Order rehydrate(String orderId) {
    List<OrderEventEntity> events = eventRepository.findByAggregateIdOrderByVersion(orderId);
    Order order = new Order();
    for (OrderEventEntity e : events) {
        order.apply(deserialize(e));   // each event mutates state: CREATED -> SHIPPED -> ...
    }
    return order;
}
```

3.2 Rebuilding current state by replaying events

```
@Table("order_summary_by_user")
public class OrderSummaryView {
    @PrimaryKeyColumn(type = PrimaryKeyType.PARTITIONED)
    private Long userId;
    @PrimaryKeyColumn(type = PrimaryKeyType.CLUSTERED, ordering = Ordering.DESCENDING)
    private String orderId;
    private String productName;   // denormalized from product-service, no join needed
    private String status;
    private BigDecimal totalPrice;
}

@Component
public class OrderProjectionUpdater {

    private final OrderSummaryRepository cassandraRepo;

    @KafkaListener(topics = "order.events", groupId = "order-query-projector")
    public void on(DomainEvent event) {
        if (event instanceof OrderCreatedEvent e) {
            cassandraRepo.save(new OrderSummaryView(e.userId(), e.orderId(), e.productName(), "CREATED",
↪   e.total()));
        } else if (event instanceof OrderShippedEvent e) {
            OrderSummaryView view = cassandraRepo.findByOrderId(e.orderId());
            view.setStatus("SHIPPED");
            cassandraRepo.save(view);
        }
    }
}
```

```
// Query side is now a completely separate, read-optimized path — no writes here.
@RestController
@RequestMapping("/api/orders/query")
```

```
public class OrderQueryController {
    private final OrderSummaryRepository cassandraRepo;

    @GetMapping("/user/{userId}")
    public List<OrderSummaryView> getOrdersForUser(@PathVariable Long userId) {
        return cassandraRepo.findByUserId(userId);   // single-partition read, sub-ms latency
    }
}
```

3.3 Read side — Materialized View in Cassandra (fast queries, denormalized)

Real-Time Usage

- **Banking ledgers**: event sourcing is the natural fit — a ledger is *literally* a sequence of immutable transactions; current balance is a projection/replay of all events, satisfying audit and regulatory requirements.
- **E-commerce order history dashboards**: CQRS read models in Cassandra/Elasticsearch power fast "My Orders" pages without hammering the transactional order DB.
- **Airline booking systems**: Sagas coordinate seat-hold → payment → ticket-issuance across services without 2PC, since airline inventory systems can't tolerate long-held distributed locks.

Architectural Flow Diagram

Q4: Microservices Distributed Caching (Caffeine, Redis, Eviction Policies, @CachePut)

Definition

Caching reduces load on databases and downstream services by storing frequently-accessed data closer to the consumer:

- **Caffeine**: in-JVM, **local** cache — extremely fast (nanoseconds), but not shared across instances.
- **Redis**: **distributed** cache — shared across all service instances, adds a network hop but stays consistent cluster-wide.
- **Eviction policies**: rules for removing entries when the cache is full or stale.
  - **LRU (Least Recently Used)**: evict the entry that hasn't been accessed in the longest time.
  - **LFU (Least Frequently Used)**: evict the entry accessed the fewest times.
  - **TTL (Time-To-Live)**: evict entries after a fixed duration, regardless of usage.
- **@Cacheable / @CachePut / @CacheEvict**: Spring annotations to read-through, write-through, and invalidate cache entries declaratively.

Hands-On: Two-Level Cache (Caffeine L1 + Redis L2) for Product Lookups

```
<dependency>
    <groupId>com.github.ben-manes.caffeine</groupId>
    <artifactId>caffeine</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
```

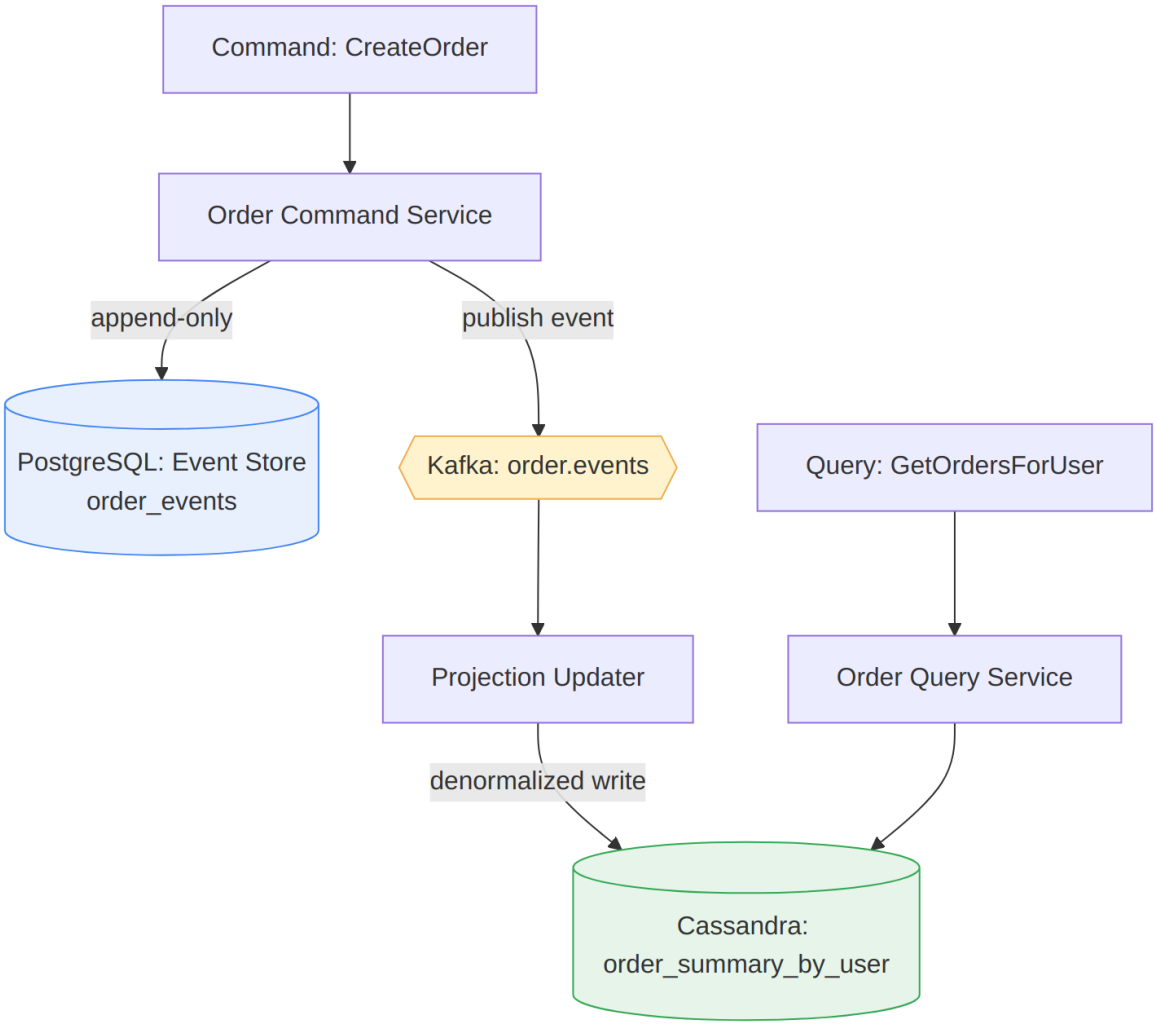


Figure 15: Architectural Flow Diagram

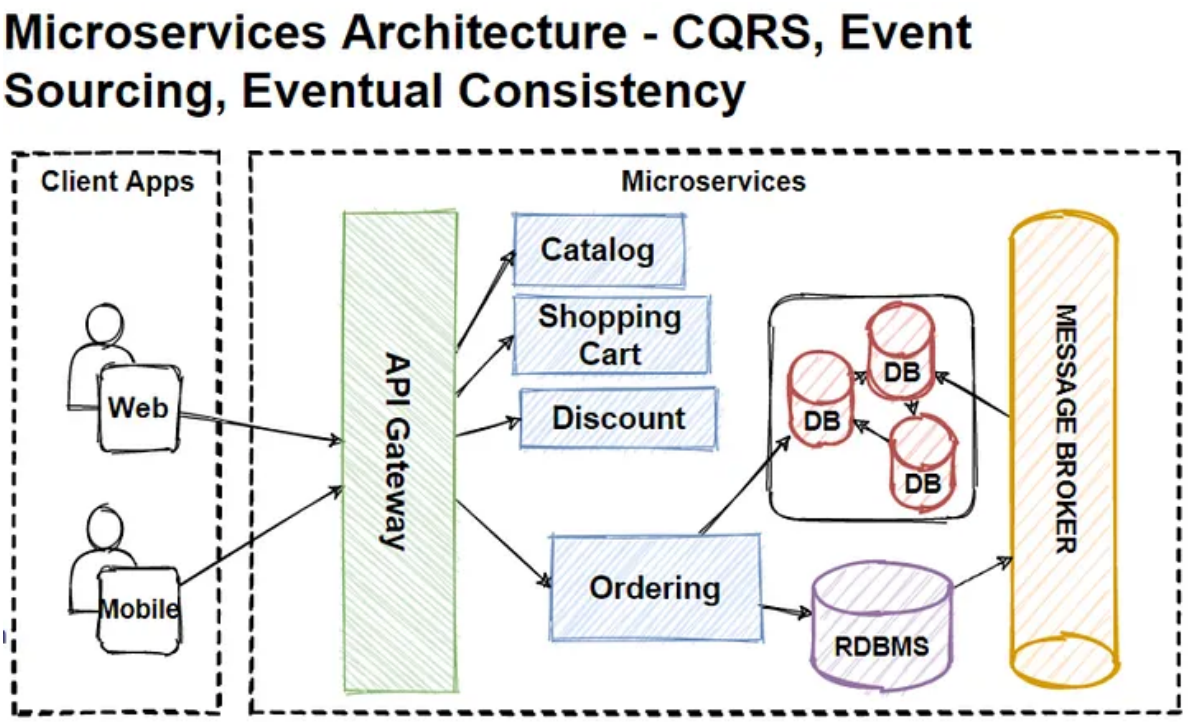


Figure 16: Microservices Architecture with CQRS, Event Sourcing, Eventual Consistency — API Gateway routing to Catalog, Shopping Cart, Discount, and Ordering, with Ordering's writes fanning out through a Message Broker to multiple read DBs

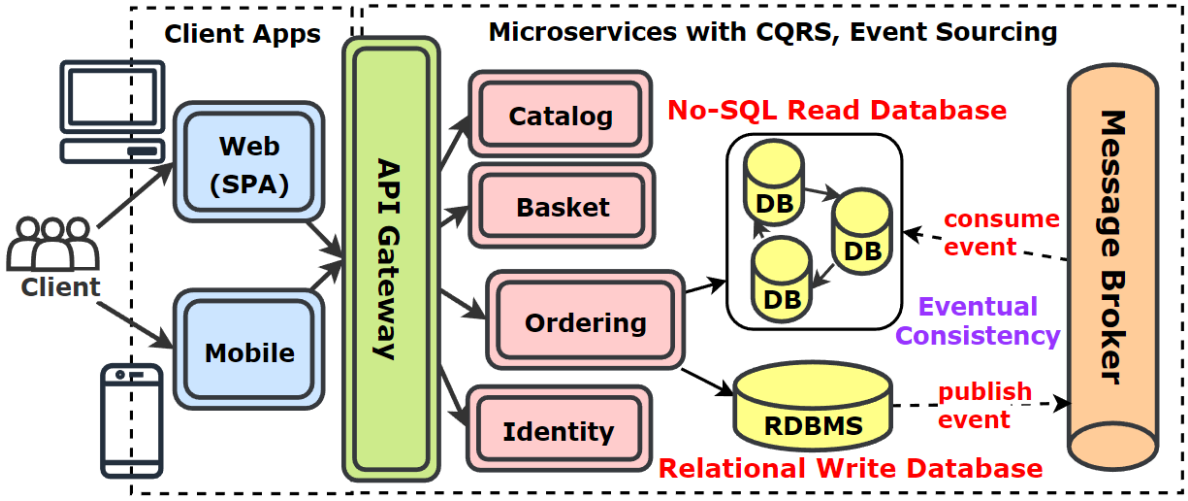


Figure 17: Detailed view — Web/Mobile clients through an API Gateway to Catalog, Basket, Ordering, and Identity services; Ordering writes to a Relational Write Database and publishes events via a Message Broker to a No-SQL Read Database, achieving Eventual Consistency

```
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

4.1 Dependencies

```
@Configuration
@EnableCaching
public class CaffeineCacheConfig {

    @Bean
    public CacheManager caffeineCacheManager() {
        CaffeineCacheManager manager = new CaffeineCacheManager("products");
        manager.setCaffeine(Caffeine.newBuilder()
                .maximumSize(10_000)                    // triggers LRU-ish eviction (window-TinyLFU internally)
                .expireAfterWrite(Duration.ofMinutes(5)) // TTL
                .recordStats());
        return manager;
    }
}
```

4.2 Caffeine (L1, local) — LRU + TTL config

```
@Bean
public RedisCacheManager redisCacheManager(RedisConnectionFactory factory) {
    RedisCacheConfiguration config = RedisCacheConfiguration.defaultCacheConfig()
        .entryTtl(Duration.ofMinutes(30))
        .serializeValuesWith(RedisSerializationContext.SerializationPair
            .fromSerializer(new GenericJackson2JsonRedisSerializer()));

    return RedisCacheManager.builder(factory)
        .cacheDefaults(config)
        .withCacheConfiguration("products", config)
        .build();
}
```

4.3 Redis (L2, distributed) — TTL config

```
@Service
public class ProductService {

    private final ProductRepository repository;

    // L1: checked first (fast, local). Falls through to method body on miss.
    @Cacheable(value = "products", cacheManager = "caffeineCacheManager", key = "#id")
    public ProductDto getProductLocalCache(Long id) {
        return getProductDistributedCache(id);
    }

    // L2: checked on L1 miss. Falls through to DB on miss.
    @Cacheable(value = "products", cacheManager = "redisCacheManager", key = "#id")
    public ProductDto getProductDistributedCache(Long id) {
        return repository.findById(id)
                .map(ProductMapper::toDto)
```

```
                .orElseThrow(() -> new ProductNotFoundException(id));
    }

    // Update DB AND refresh cache atomically (write-through) — avoids stale-cache window
    @CachePut(value = "products", cacheManager = "redisCacheManager", key = "#dto.id")
    public ProductDto updateProduct(ProductDto dto) {
        Product entity = repository.save(ProductMapper.toEntity(dto));
        return ProductMapper.toDto(entity);
    }

    // Invalidate cache on delete — evict from BOTH levels
    @Caching(evict = {
        @CacheEvict(value = "products", cacheManager = "caffeineCacheManager", key = "#id"),
        @CacheEvict(value = "products", cacheManager = "redisCacheManager", key = "#id")
    })
    public void deleteProduct(Long id) {
        repository.deleteById(id);
    }
}
```

4.4 Service layer with cache-aside pattern

```
# redis.conf — cluster-wide eviction policy
maxmemory 2gb
maxmemory-policy allkeys-lfu   # evict least-frequently-used keys when memory limit hit
```

4.5 LFU-style eviction for a “hot products” cache (Redis with maxmemory-policy)

4.6 Dedicated Hands-On: LRU, LFU, and TTL — Standalone Implementations The examples above show LRU/LFU/TTL wired into Caffeine/Redis configs. Here’s each policy in isolation, including a from-scratch implementation, so the eviction *mechanics* are clear on their own.

LRU (Least Recently Used) — evict the entry not accessed in the longest time

```
// Classic LRU using LinkedHashMap's access-order mode — textbook implementation
public class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    public LruCache(int capacity) {
        super(capacity, 0.75f, true);   // accessOrder=true -> reorders entries on every get()
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;       // evict the least-recently-used entry once over capacity
    }
}

// Usage
LruCache<Long, ProductDto> cache = new LruCache<>(1000);
cache.put(1L, product1);
cache.get(1L);            // touching 1L marks it as most-recently-used
cache.put(1001L, product1001);   // if now over capacity, the LEAST-recently-touched entry is evicted first
```

```
# Redis — native, cluster-wide LRU eviction
maxmemory 2gb
maxmemory-policy allkeys-lru
```

```
// Caffeine — size-bounded cache (Window-TinyLFU under the hood, but same "evict oldest-unused-first" contract)
Cache<Long, ProductDto> lruLikeCache = Caffeine.newBuilder()
    .maximumSize(1000)
    .build();
```

LFU (Least Frequently Used) — evict the entry accessed the fewest times

```
# Redis — native, cluster-wide LFU eviction
maxmemory 2gb
maxmemory-policy allkeys-lfu
```

```
# Inspect Redis's internal per-key LFU frequency counter (0-255, decays over time)
redis-cli OBJECT FREQ product:1042
```

```
// Custom O(1) LFU cache — frequency-bucket algorithm (each bucket = set of keys with that hit-count)
public class LfuCache<K, V> {
    private final int capacity;
    private final Map<K, V> values = new HashMap<>();
    private final Map<K, Integer> frequencies = new HashMap<>();
    private final Map<Integer, LinkedHashSet<K>> frequencyBuckets = new HashMap<>();
    private int minFrequency = 0;

    public LfuCache(int capacity) { this.capacity = capacity; }

    public V get(K key) {
        if (!values.containsKey(key)) return null;
        bumpFrequency(key);
        return values.get(key);
    }

    public void put(K key, V value) {
        if (capacity <= 0) return;
        if (values.containsKey(key)) {
            values.put(key, value);
            bumpFrequency(key);
            return;
        }
        if (values.size() >= capacity) {
            LinkedHashSet<K> bucket = frequencyBuckets.get(minFrequency);
            K evictKey = bucket.iterator().next();      // least-frequently-used; oldest wins on ties
            bucket.remove(evictKey);
            values.remove(evictKey);
            frequencies.remove(evictKey);
        }
        values.put(key, value);
        frequencies.put(key, 1);
        frequencyBuckets.computeIfAbsent(1, k -> new LinkedHashSet<>()).add(key);
        minFrequency = 1;
    }

    private void bumpFrequency(K key) {
        int freq = frequencies.get(key);
        frequencyBuckets.get(freq).remove(key);
        if (frequencyBuckets.get(freq).isEmpty() && minFrequency == freq) minFrequency++;
```

```
        frequencies.put(key, freq + 1);
        frequencyBuckets.computeIfAbsent(freq + 1, k -> new LinkedHashSet<>()).add(key);
    }
}
```

TTL (Time-To-Live) — evict after a fixed duration, regardless of usage

```
// Caffeine — two TTL variants
Cache<Long, ProductDto> ttlCache = Caffeine.newBuilder()
    .expireAfterWrite(Duration.ofMinutes(5))     // fixed TTL from write time
    .build();

Cache<Long, ProductDto> slidingTtlCache = Caffeine.newBuilder()
    .expireAfterAccess(Duration.ofMinutes(2))    // TTL resets on every read (sliding expiration)
    .build();
```

```
# Redis — native TTL commands
SET product:1042 '{"name":"Widget"}' EX 1800   # key expires in 1800 seconds
TTL product:1042                                # check remaining time-to-live
PERSIST product:1042                            # remove the TTL, make the key permanent
```

```
// Spring RedisCacheManager — different TTL per cache name
@Bean
public RedisCacheManager redisCacheManager(RedisConnectionFactory factory) {
    Map<String, RedisCacheConfiguration> perCacheConfigs = new HashMap<>();
    perCacheConfigs.put("products",
↪   RedisCacheConfiguration.defaultCacheConfig().entryTtl(Duration.ofMinutes(30)));
    perCacheConfigs.put("sessions",
↪   RedisCacheConfiguration.defaultCacheConfig().entryTtl(Duration.ofMinutes(15)));

    return RedisCacheManager.builder(factory)
        .withInitialCacheConfigurations(perCacheConfigs)
        .build();
}
```

```
// Manual TTL cache with a background sweeper — for when you're not using Caffeine/Redis at all
public class TtlCache<K, V> {
    private record Entry<V>(V value, long expiresAtMillis) {}
    private final Map<K, Entry<V>> store = new ConcurrentHashMap<>();
    private final ScheduledExecutorService sweeper = Executors.newSingleThreadScheduledExecutor();

    public TtlCache(long sweepIntervalMs) {
        sweeper.scheduleAtFixedRate(this::evictExpired, sweepIntervalMs, sweepIntervalMs,
↪   TimeUnit.MILLISECONDS);
    }

    public void put(K key, V value, Duration ttl) {
        store.put(key, new Entry<>(value, System.currentTimeMillis() + ttl.toMillis()));
    }

    public V get(K key) {
        Entry<V> e = store.get(key);
        if (e == null) return null;
        if (System.currentTimeMillis() > e.expiresAtMillis()) {
            store.remove(key);   // lazy eviction on read, in addition to the background sweep
            return null;
        }
        return e.value();
```

```
    }

    private void evictExpired() {
        long now = System.currentTimeMillis();
        store.entrySet().removeIf(e -> now > e.getValue().expiresAtMillis());
    }
}
```

Real-Time Usage

- **Product catalog pages (e-commerce)**: product details are read millions of times per write — Caffeine absorbs the "hot 1000 SKUs" locally per instance, Redis absorbs the long tail cluster-wide, cutting DB load by 90%+.
- **Session/token caches**: Redis is the default for storing auth session tokens across horizontally-scaled service instances — no instance-local state, works behind any load balancer.
- **Rate limiting counters**: Redis with TTL-based keys implements sliding-window rate limits shared across every gateway instance.
- **Recommendation results**: LFU eviction fits recommendation caches well — "popular for everyone" items should stay resident longer than one-off niche lookups.

Architectural Flow Diagram

Q5: Microservices Deployment — Containers, Orchestrators, CI/CD, Canary, Blue-Green

Definition

- **Docker**: packages a service and its runtime dependencies into a portable, immutable **container image**.
- **Kubernetes (K8s)**: orchestrates containers across a cluster — handles scheduling, self-healing, auto-scaling, and service discovery.
- **CI/CD Pipeline**: automated build → test → package → deploy flow triggered on every commit.
- **Canary Deployment**: roll out a new version to a **small percentage** of traffic first, monitor error rates/latency, then gradually increase — limits blast radius of a bad release.
- **Blue-Green Deployment**: run two full production environments (**Blue** = current, **Green** = new); switch all traffic at once via a router/load-balancer flip — enables instant rollback by flipping back.

	Canary	Blue-Green
Traffic shift	Gradual (5% → 25% → 100%)	Instant (0% → 100%)
Blast radius	Small, contained	All-or-nothing (but reversible in seconds)
Infra cost	Runs alongside old version at partial scale	Requires 2x full environments temporarily
Best for	High-traffic services needing gradual validation	Services where instant, clean cutover/rollback matters

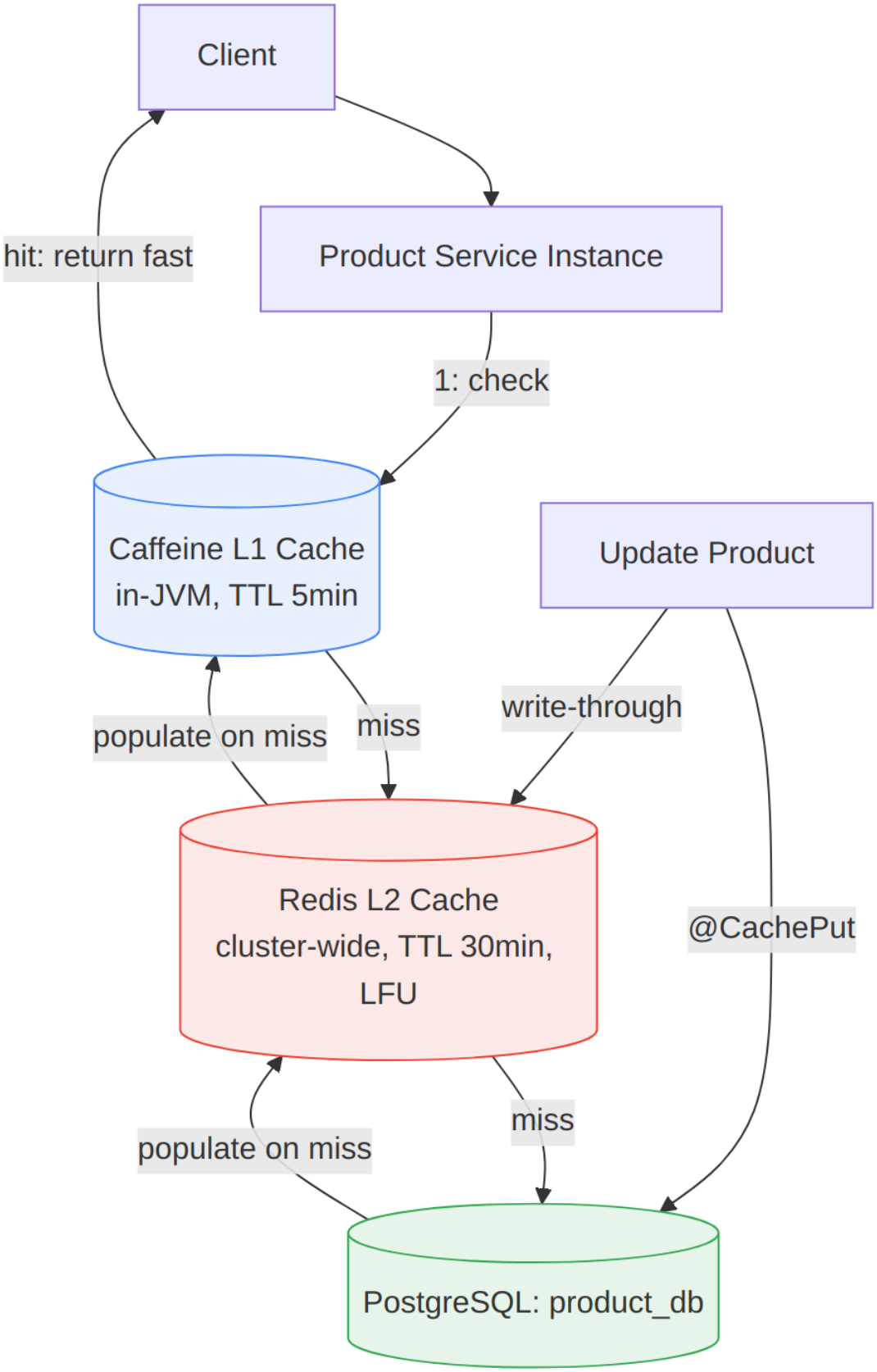


Figure 18: Architectural Flow Diagram

Hands-On

```
# --- Build stage ---
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN --mount=type=cache,target=/root/.m2 mvn -B clean package -DskipTests

# --- Runtime stage (small image) ---
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/order-service-*.jar app.jar
EXPOSE 8083
ENTRYPOINT ["java", "-jar", "app.jar"]
```

5.1 Dockerfile (multi-stage build for Spring Boot)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service-v1
  labels: { app: order-service, version: v1 }
spec:
  replicas: 5
  selector:
    matchLabels: { app: order-service, version: v1 }
  template:
    metadata:
      labels: { app: order-service, version: v1 }
    spec:
      containers:
        - name: order-service
          image: myregistry/order-service:1.4.0
          ports: [{ containerPort: 8083 }]
          resources:
            requests: { cpu: "250m", memory: "512Mi" }
            limits: { cpu: "500m", memory: "1Gi" }
          readinessProbe:
            httpGet: { path: /actuator/health/readiness, port: 8083 }
          livenessProbe:
            httpGet: { path: /actuator/health/liveness, port: 8083 }
---
apiVersion: v1
kind: Service
metadata:
  name: order-service
spec:
  selector:
    app: order-service   # note: NOT pinned to a version yet — used for Blue-Green cutover
  ports:
    - port: 80
      targetPort: 8083
```

5.2 Kubernetes Deployment + Service

```yaml
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: order-service
spec:
  hosts: [order-service]
  http:
    - route:
        - destination: { host: order-service, subset: v1 }
          weight: 90        # 90% stable
        - destination: { host: order-service, subset: v2 }
          weight: 10        # 10% canary
---
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: order-service
spec:
  host: order-service
  subsets:
    - name: v1
      labels: { version: v1 }
    - name: v2
      labels: { version: v2 }
```

```bash
# Gradually shift traffic once canary metrics look healthy (error rate, p99 latency)
kubectl patch virtualservice order-service --type merge -p \
  '{"spec":{"http":[{"route":[{"destination":{"host":"order-
  ↪  service","subset":"v1"},"weight":50},{"destination":{"host":"order-
  ↪  service","subset":"v2"},"weight":50}]}]}}'
# ... eventually 0/100, then delete the v1 Deployment
```

5.3 Canary rollout with Istio VirtualService (weighted traffic split)

```bash
# Both "blue" (v1, live) and "green" (v2, staged) run simultaneously, fully scaled
kubectl apply -f order-service-green-deployment.yaml   # deploy v2 alongside v1, 0 live traffic

# Smoke-test green directly via its own internal service/port before cutover
kubectl port-forward svc/order-service-green 8090:80

# Instant cutover: flip the main Service's selector from version=v1 to version=v2
kubectl patch service order-service -p \
  '{"spec":{"selector":{"app":"order-service","version":"v2"}}}'

# Instant rollback if something's wrong — flip the selector back
kubectl patch service order-service -p \
  '{"spec":{"selector":{"app":"order-service","version":"v1"}}}'
```

5.4 Blue-Green cutover (simple Service selector flip — no service mesh needed)

```yaml
name: order-service-ci-cd
on:
  push:
```

```yaml
    branches: [main]

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '21', distribution: 'temurin' }
      - run: mvn -B clean verify   # unit + integration tests

  build-and-push-image:
    needs: build-test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: |
          docker build -t myregistry/order-service:${{ github.sha }} .
         echo "${{ secrets.REGISTRY_PASSWORD }}" | docker login -u ${{ secrets.REGISTRY_USER }} --password-stdin
          docker push myregistry/order-service:${{ github.sha }}

  deploy-canary:
    needs: build-and-push-image
    runs-on: ubuntu-latest
    steps:
      - uses: azure/setup-kubectl@v4
      - run: |
          kubectl set image deployment/order-service-v2 order-service=myregistry/order-service:${{ github.sha }}
          kubectl apply -f k8s/canary-virtualservice-10pct.yaml
      - name: Wait & monitor canary metrics
        run: ./scripts/check-canary-health.sh   # queries Prometheus for error-rate/p99 threshold
      - name: Promote to 100% if healthy
        if: success()
        run: kubectl apply -f k8s/canary-virtualservice-100pct.yaml
      - name: Auto-rollback if unhealthy
        if: failure()
        run: kubectl apply -f k8s/canary-virtualservice-0pct.yaml
```

5.5 CI/CD Pipeline (GitHub Actions) — build, test, containerize, canary-deploy

Real-Time Usage

- **Google/Netflix/Facebook**: canary releases are the default rollout strategy for every internal service — automated health checks gate the traffic ramp-up so bad deploys are caught at 1-5% blast radius, not 100%.
- **Payment gateways (Stripe, banks)**: prefer **Blue-Green** because instant, atomic roll-back is critical — a payment bug can't be allowed to linger even at 5% traffic for the minutes it takes a canary to auto-halt.
- **Kubernetes + Helm + ArgoCD (GitOps)**: widely used in enterprises to declaratively manage canary/blue-green rollouts as Git-committed manifests, giving a full audit trail of every production change.

Architectural Flow Diagram

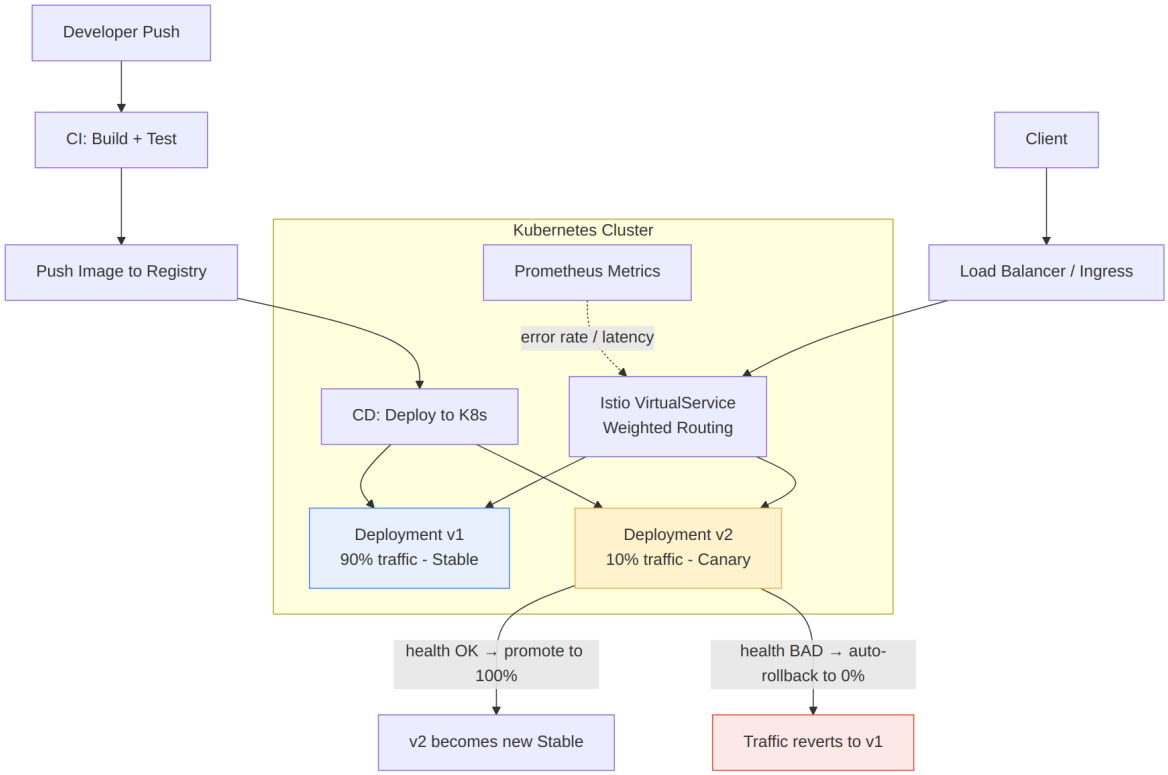


Figure 19: Architectural Flow Diagram

Coming Next — Part IV

Observability Deep Dive: centralized **Logs** (ELK / Loki + Fluentd), **Metrics** (Prometheus + Grafana dashboards), **Distributed Tracing** (OpenTelemetry + Jaeger/Zipkin) tying every microservice, Kafka topic, and cache layer built so far into a single observable system — plus alerting and SLO/SLA design. -e

Part IV — 3-Tier • API Gateway/BFF • Registry/Aggregator • Broker • Sharding • Event-Driven (Outbox) • Caching • Containers/CI-CD • Resilience4j

> *Note: A few topics below (Microservices overview, Distributed Caching, Containers+CI/CD, Event-Driven) were already deep-dived with full hands-on in **Part II** and **Part III**. To avoid pure repetition, those sections here give a tighter recap plus a **new angle/technique** not yet covered, and point back to the earlier part for the original hands-on.*

Q1: 3-Tier Architecture

Definition

3-Tier Architecture separates an application into three **logical (and often physical) layers**:

Tier	Responsibility
Presentation Tier	UI / API surface — web app, mobile app, or REST controllers
Application (Logic) Tier	Business rules, validation, orchestration
Data Tier	Persistence — database, file storage

This is **not** the same axis as microservices: 3-tier is about **separating concerns within a system** (often even within a single deployable), while microservices is about **splitting by business capability** into independently deployable units. A single microservice is often *internally* structured as 3-tier (Controller → Service → Repository).

Hands-On: Classic 3-Tier Deployment (physically separated)

```
Tier 1 (Presentation) : React SPA served from Nginx / CDN
Tier 2 (Application)  : Spring Boot REST API (stateless, horizontally scaled)
Tier 3 (Data)         : PostgreSQL (primary + read replica)
```

```java
// Presentation-facing contract stays in the Application tier as a Controller
@RestController
@RequestMapping("/api/products")
public class ProductController {
    private final ProductService service;   // -> Application tier
    public ProductController(ProductService service) { this.service = service; }

    @GetMapping("/{id}")
    public ProductDto get(@PathVariable Long id) { return service.getProduct(id); }
}

// Application tier: business logic, no SQL, no HTTP details
@Service
public class ProductService {
    private final ProductRepository repository;   // -> Data tier
    public ProductService(ProductRepository repository) { this.repository = repository; }

    public ProductDto getProduct(Long id) {
        Product p = repository.findById(id).orElseThrow(() -> new ProductNotFoundException(id));
        if (p.isDiscontinued()) throw new ProductUnavailableException(id);  // business rule lives HERE
```

```
        return ProductMapper.toDto(p);
    }
}

// Data tier: pure persistence, no business rules
public interface ProductRepository extends JpaRepository<Product, Long> { }
```

```
# Nginx (Presentation tier) — serves static SPA + reverse-proxies API calls to Tier 2
server {
    listen 443 ssl;
    location /            { root /var/www/spa; try_files $uri /index.html; }
    location /api/        { proxy_pass http://app-tier-lb:8080; }
}
```

Real-Time Usage

- **Classic enterprise web portals** (HR systems, banking web portals, government e-services) — predictable, well-understood, easy to secure (each tier gets its own network zone / firewall rules, e.g. DB tier never directly reachable from the internet).
- **Small-to-mid SaaS products** before they need microservices — a well-factored 3-tier app scales a long way with just horizontal scaling of the app tier and a read-replica on the data tier.

Architectural Flow Diagram

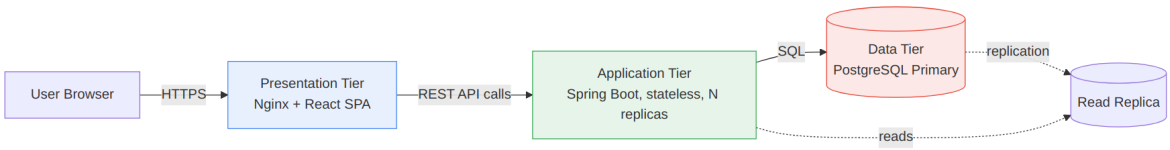


Figure 20: Architectural Flow Diagram

Q2: Microservices Architectures (System Overview)

Definition — Recap

Microservices structure a system as a set of small, independently deployable services, each owning its data, communicating over the network, aligned to business capabilities. *(Full definition, decomposition strategies, and per-service hands-on: see Part I §Monolith baseline, Part II Q2, and Part III Q1.)*

Hands-On: Wiring the Whole System Together (docker-compose)

This ties together every piece built across Parts I–III into one runnable local environment:

```
version: '3.8'
services:
  api-gateway:      { build: ./gateway,         ports: ["8080:8080"] }
  user-service:     { build: ./user-service,    ports: ["8081:8081"], depends_on: [postgres-user] }
  product-service:  { build: ./product-service, ports: ["8082:8082"], depends_on: [postgres-product, redis] }
  order-service:    { build: ./order-service,   ports: ["8083:8083"], depends_on: [cassandra, kafka] }
```

```
postgres-user:    { image: postgres:16, environment: { POSTGRES_DB: user_db } }
postgres-product: { image: postgres:16, environment: { POSTGRES_DB: product_db } }
cassandra:        { image: cassandra:5.0 }
redis:            { image: redis:7-alpine }
kafka:            { image: confluentinc/cp-kafka:7.6.0 }
zookeeper:        { image: confluentinc/cp-zookeeper:7.6.0 }
```

Real-Time Usage

- Any mid-to-large e-commerce, ride-hailing, fintech, or streaming platform that needs independent scaling and independent release cadence across teams (Amazon, Uber, Netflix, Spotify — recap from Part I/II).

Architectural Flow Diagram

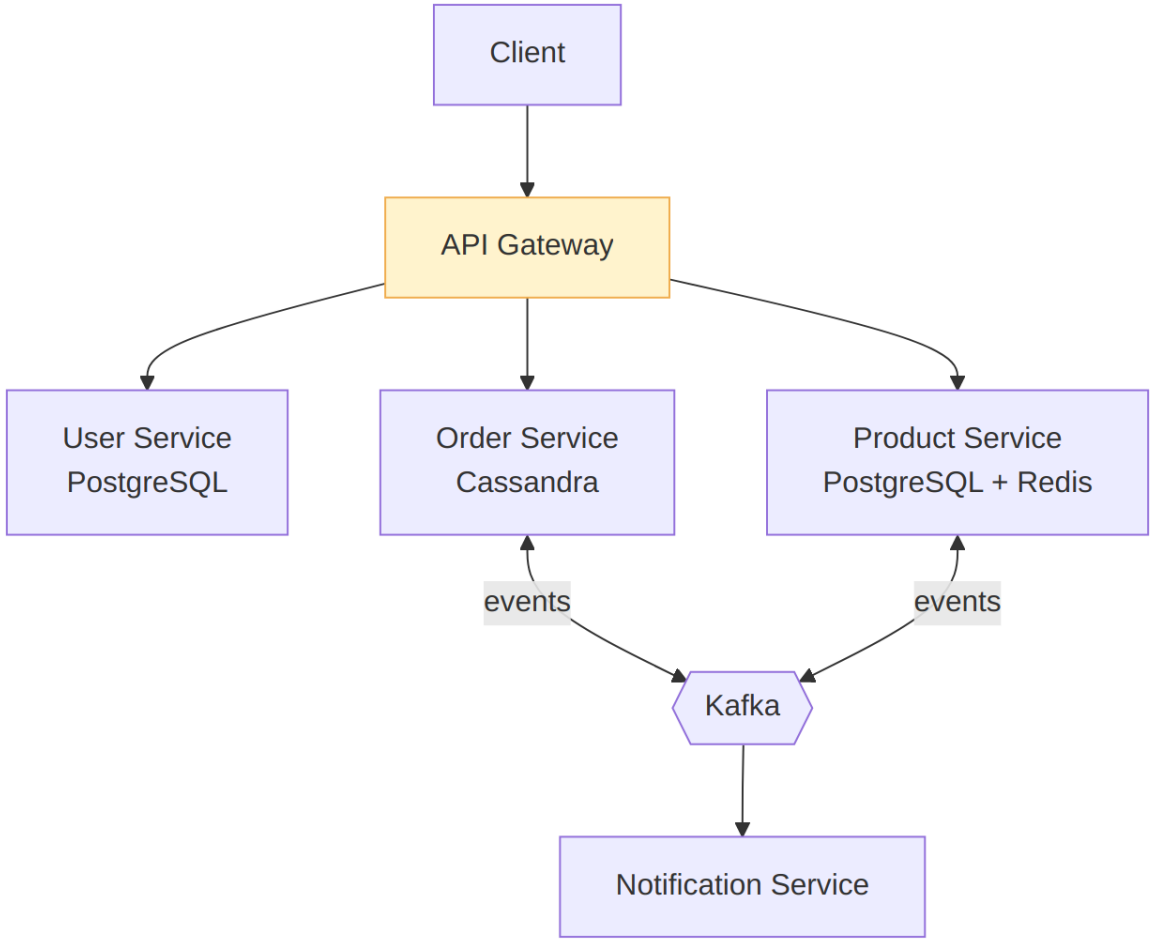


Figure 21: Architectural Flow Diagram

Q3: Microservices Architecture — API Gateways (Deep Dive)

Definition

The **API Gateway** pattern centralizes cross-cutting concerns so individual services don't reimplement them: **routing, authentication/authorization, rate limiting, request/response transformation, protocol translation (REST↔gRPC), and observability (correlation IDs)**.

Since the gateway sits between clients and the internal microservices, it's the natural place to implement several useful, reusable features:

1. Reverse Proxy / Gateway Routing — the API Gateway redirects incoming requests to the correct internal microservice endpoint, typically using **Layer 7 (HTTP) routing**. This decouples client applications from internal microservices at the network layer: clients only ever know the gateway's address. It also **abstracts internal operations** — backend services can change freely (be split, merged, re-hosted) without client applications ever needing to update, since the gateway's external contract stays stable.

2. Requests Aggregation — the gateway can combine calls to **multiple internal microservices into a single client request/response**. The client sends one request; the gateway fans it out to several backend services, aggregates the results, and returns one combined response. This is the main tool for reducing **chatty communication** between clients and the backend (recap: the Aggregator pattern in Q5).

3. Cross-Cutting Concerns / Gateway Offloading — since the gateway handles every request in one centralized place, it's best practice to implement cross-cutting functionality there instead of duplicating it in every service: - Authentication and authorization - Service discovery integration - Response caching - Retry policies, circuit breaker, and Quality of Service (QoS) - Rate limiting and throttling - Load balancing - Logging, tracing, and correlation - Headers, query strings, and claims transformation - IP allowlisting

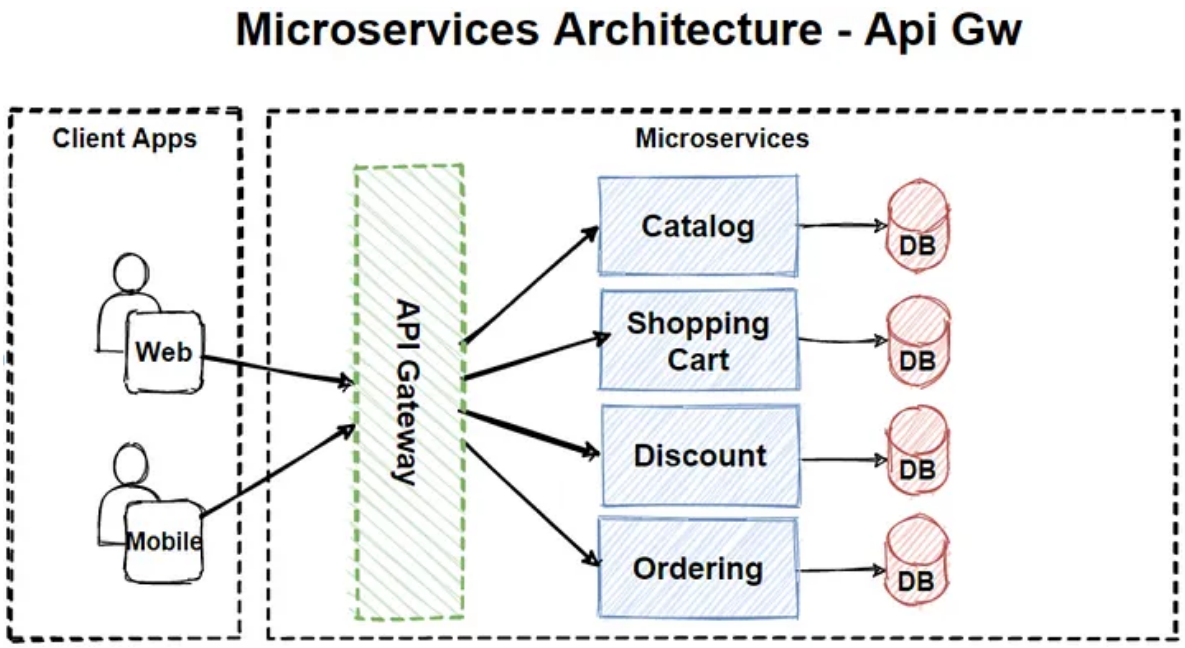


Figure 22: Microservices Architecture with an API Gateway — Web and Mobile clients route through a single API Gateway to Catalog, Shopping Cart, Discount, and Ordering services, each with its own DB

For reference, here's the feature set of **Ocelot** (a popular .NET API Gateway), which maps directly onto the concerns above:

Ocelot Features	
Routing	Authentication
Request Aggregation	Authorization
Service Discovery with Consul & Eureka	Throttling
Load Balancing	Logging, Tracing
Correlation Pass-Through	Headers/Query String Transformation
Quality of Service	Custom Middleware

Figure 23: Ocelot Features — Routing, Request Aggregation, Service Discovery with Consul & Eureka, Load Balancing, Correlation Pass-Through, Quality of Service, Authentication, Authorization, Throttling, Logging/Tracing, Headers/Query String Transformation, and Custom Middleware

A caution — single point of failure: putting *all* client applications behind one API Gateway is a great start, but it comes with a risk. As the number of client types grows, or as more business logic accumulates inside the gateway, that single gateway becomes both a **single point of failure** and, eventually, an **anti-pattern** (a shared "God gateway" doing too much for too many different clients with different needs). The fix is the **BFF (Backend For Frontend) pattern** — see Q4 next — which splits one shared gateway into a dedicated gateway per client type.

Hands-On: Spring Cloud Gateway with JWT Auth + Rate Limiting

```java
@Component
public class JwtAuthFilter implements GlobalFilter, Ordered {
    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String token = exchange.getRequest().getHeaders().getFirst("Authorization");
        if (token == null || !jwtValidator.isValid(token)) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }
        // propagate a correlation id downstream for distributed tracing
        exchange.getRequest().mutate().header("X-Correlation-Id", UUID.randomUUID().toString());
        return chain.filter(exchange);
    }
    @Override public int getOrder() { return -1; }  // run before routing
}
```

```yaml
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service          # load-balanced via service registry
          predicates: [Path=/api/orders/**]
          filters:
```

```
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 100   # tokens/sec (token-bucket algorithm)
                redis-rate-limiter.burstCapacity: 200
            - name: CircuitBreaker
              args: { name: orderCB, fallbackUri: forward:/fallback/orders }
```

Real-Time Usage

- Every mobile-app backend (banking apps, ride-hailing apps) uses a gateway so client apps never talk to internal services directly — internal topology can change freely without client updates.
- Rate limiting at the gateway (not per-service) protects the *whole* backend from a single noisy client/API key.

Architectural Flow Diagram

Q4: Microservices Architecture — API Gateway BFF (Backend For Frontend)

Definition

BFF creates a **dedicated gateway per client type** (web, mobile, smart-TV) instead of one generic gateway for all clients. Each BFF shapes/aggregates responses exactly the way that client needs — mobile gets a slim payload to save bandwidth/battery; web gets a richer payload for a feature-dense dashboard.

Hands-On: mobile-bff vs web-bff aggregating the same data differently

```
// mobile-bff: minimal payload, optimized for small screens & slow networks
@RestController
@RequestMapping("/mobile/order-summary/{orderId}")
public class MobileOrderBff {
    private final OrderClient orderClient;
    private final ProductClient productClient;

    @GetMapping
    public MobileOrderView get(@PathVariable String orderId) {
        Order order = orderClient.get(orderId);
        String productName = productClient.getName(order.getProductId()); // name only
        return new MobileOrderView(order.getId(), productName, order.getStatus()); // 3 fields
    }
}

// web-bff: rich payload for a full dashboard
@RestController
@RequestMapping("/web/order-summary/{orderId}")
public class WebOrderBff {
    private final OrderClient orderClient;
    private final ProductClient productClient;
    private final UserClient userClient;
    private final ShippingClient shippingClient;
```

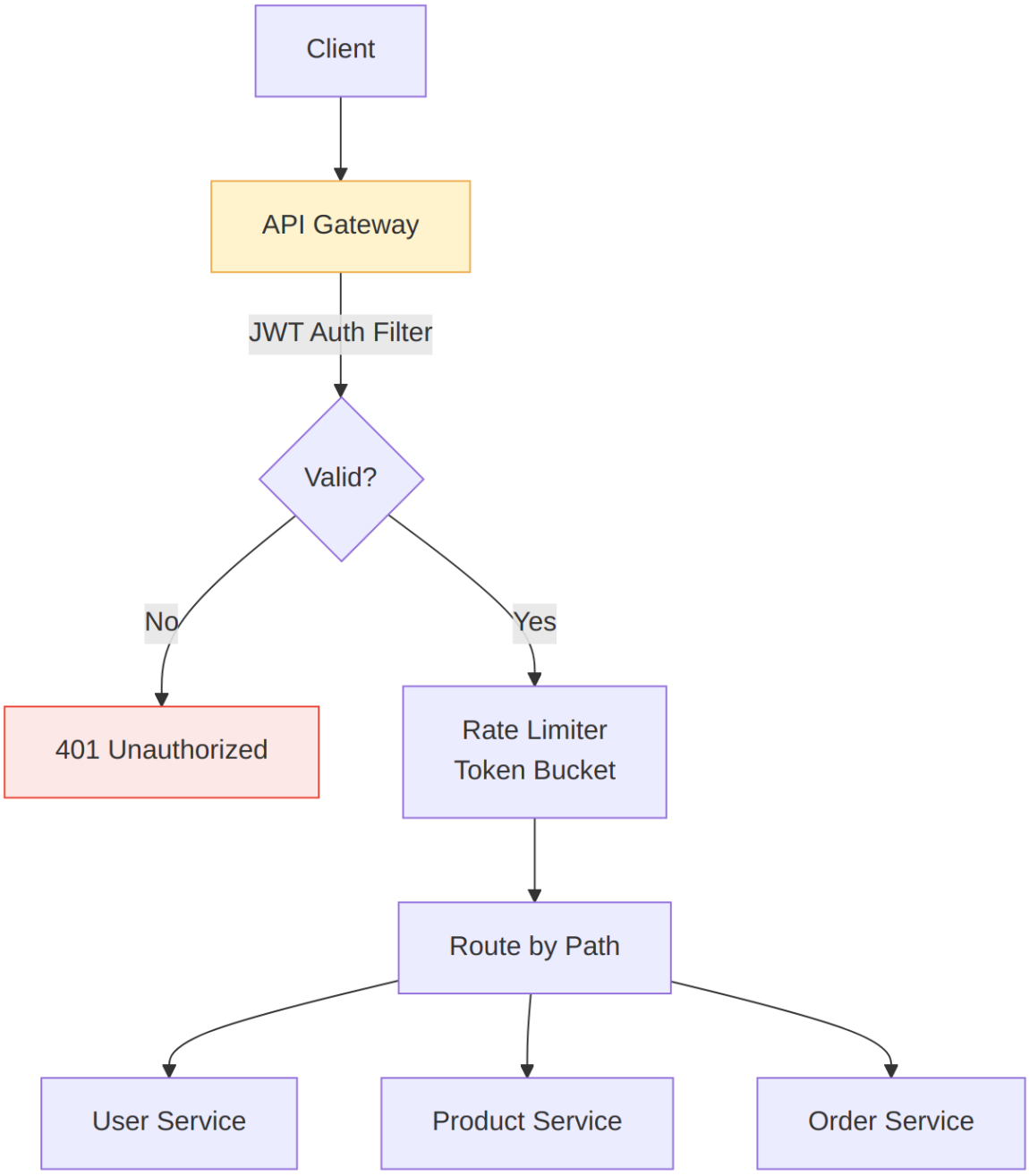


Figure 24: Architectural Flow Diagram

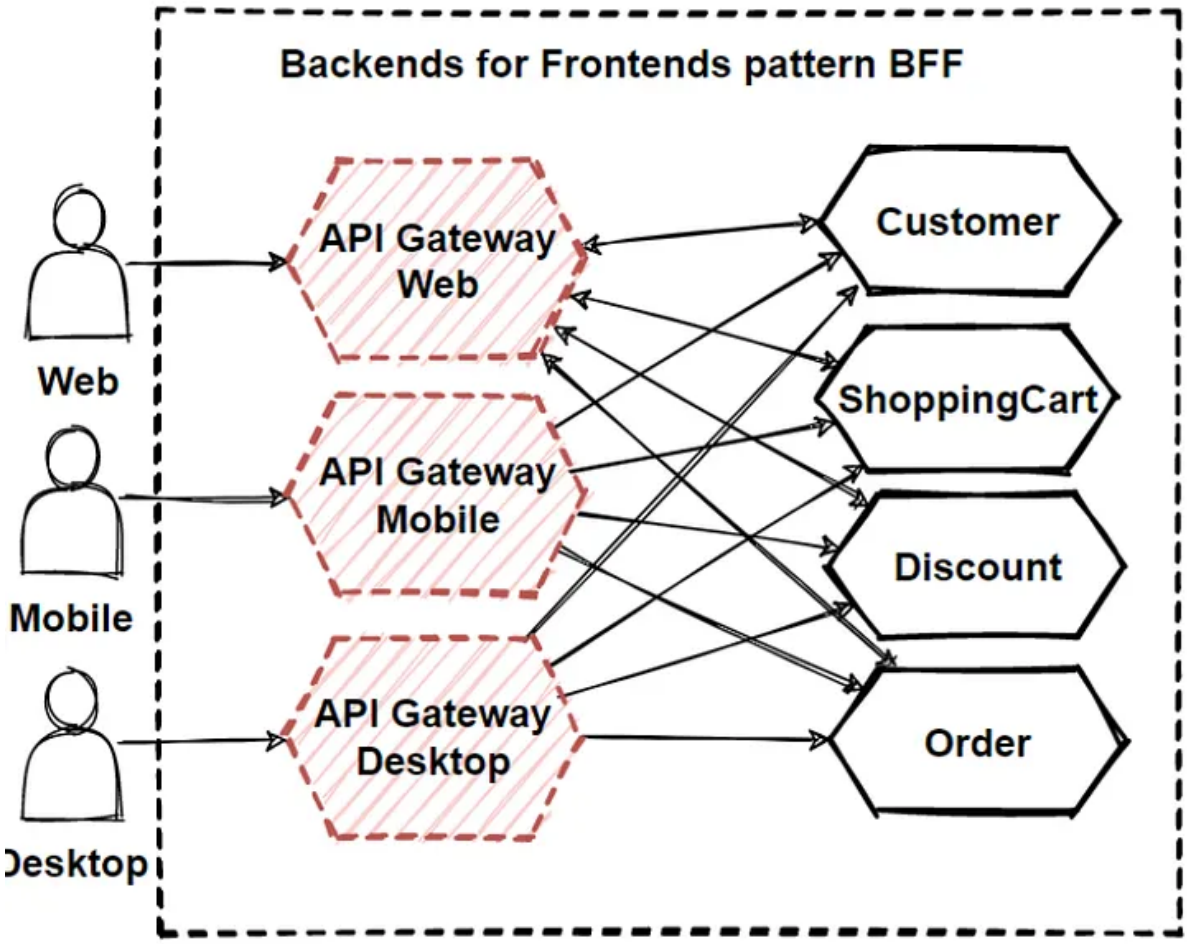


Figure 25: Backends for Frontends (BFF) Pattern — separate API Gateway Web, API Gateway Mobile, and API Gateway Desktop, each calling into the shared Customer, ShoppingCart, Discount, and Order services

```
    @GetMapping
    public WebOrderView get(@PathVariable String orderId) {
        Order order = orderClient.get(orderId);
        return new WebOrderView(
            order,
            productClient.getFullDetails(order.getProductId()),  // images, specs, reviews
            userClient.getProfile(order.getUserId()),
            shippingClient.getTrackingHistory(orderId)
        );
    }
}
```

Real-Time Usage

- **Netflix**: pioneered BFF because TV, mobile, and web UIs needed drastically different data shapes and couldn't share one gateway contract without constant compromise.
- **SoundCloud, Spotify**: mobile BFFs strip down catalog metadata heavily to save data usage; web/desktop BFFs return full metadata for richer UI.

Architectural Flow Diagram

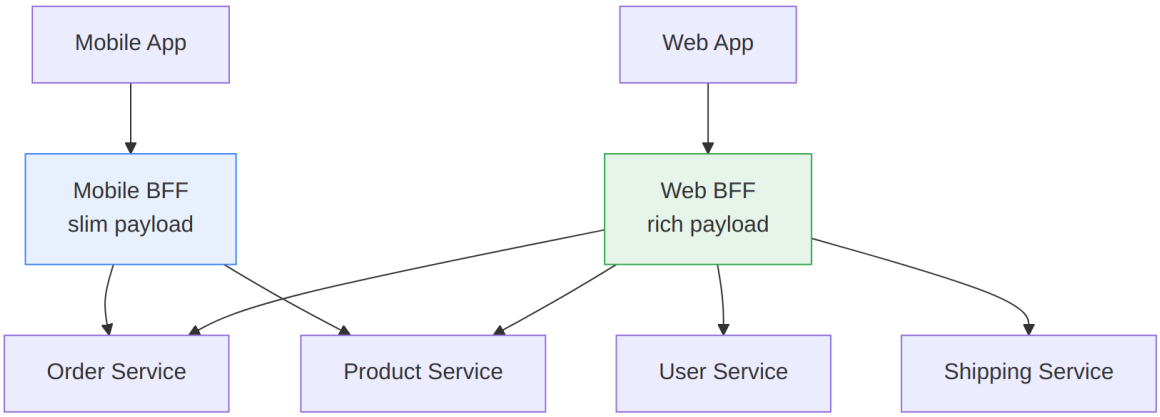


Figure 26: Architectural Flow Diagram

Q5: Microservices Architecture — Service Aggregator / Registry Patterns

Definition

- **Service Registry** (Eureka, Consul, Kubernetes DNS): services **register themselves** on startup; callers **discover** healthy instances dynamically instead of hardcoding IPs — essential once instances scale up/down or get rescheduled.
- **Aggregator Pattern**: a component (gateway filter or dedicated service) calls **multiple backend services** and **combines** results into one response, so the client makes one call instead of many.

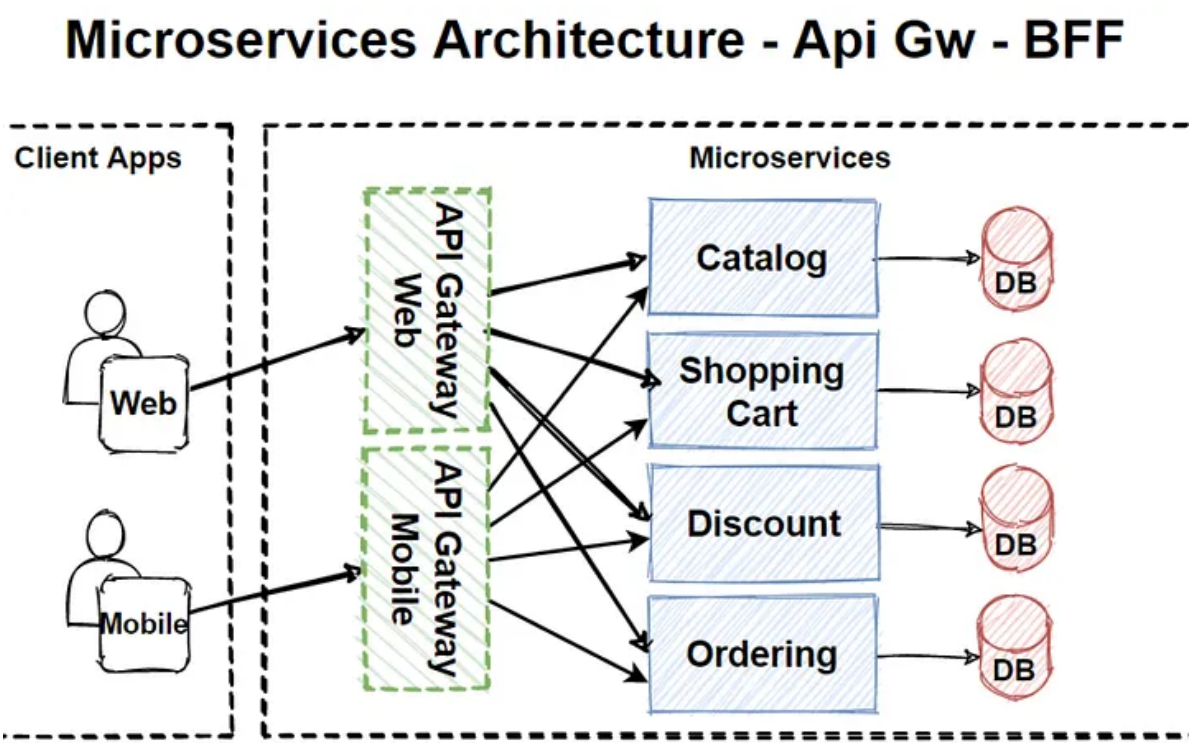


Figure 27: Microservices Architecture with API Gateway BFF — separate Web and Mobile API Gateways routing into Catalog, Shopping Cart, Discount, and Ordering microservices, each with its own DB

Hands-On: Eureka Registry + a Dedicated Aggregator Service

```
// Eureka Server
@SpringBootApplication
@EnableEurekaServer
public class DiscoveryServerApplication { }
```

```
# Each microservice registers itself
spring:
  application:
    name: order-service
eureka:
  client:
    service-url: { defaultZone: http://discovery-server:8761/eureka }
  instance:
    prefer-ip-address: true
```

```
// Dedicated Aggregator microservice — calls 3 services concurrently, merges results
@Service
public class OrderAggregatorService {

    private final WebClient.Builder webClientBuilder;  // load-balanced via Eureka ("lb://")

    public Mono<OrderFullView> aggregate(String orderId) {
        Mono<Order> order = webClientBuilder.build().get()
            .uri("lb://order-service/api/orders/{id}", orderId).retrieve().bodyToMono(Order.class);
        Mono<ProductDto> product = order.flatMap(o -> webClientBuilder.build().get()
```

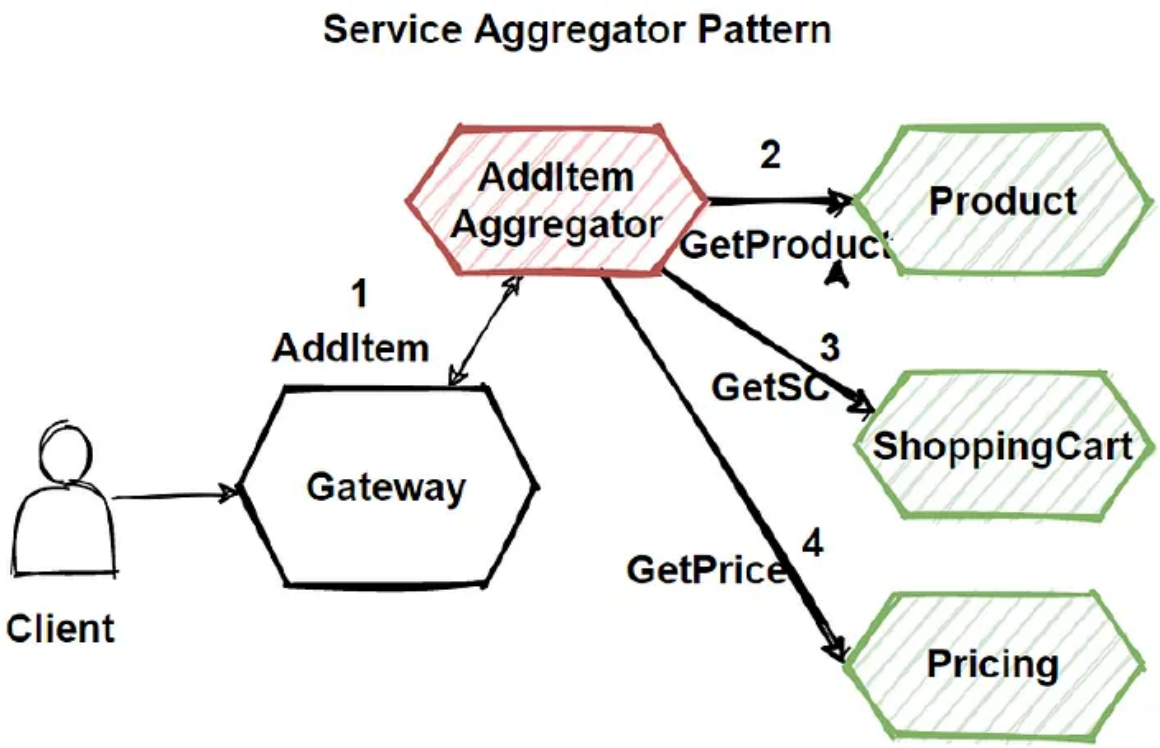


Figure 28: Service Aggregator Pattern — Client calls Gateway (1: AddItem), which delegates to an AddItem Aggregator that fans out to Product (2: GetProduct), ShoppingCart (3: GetSC), and Pricing (4: GetPrice)

```
            .uri("lb://product-service/api/products/{id}",
            ↪  o.getProductId()).retrieve().bodyToMono(ProductDto.class));
        Mono<UserDto> user = order.flatMap(o -> webClientBuilder.build().get()
            .uri("lb://user-service/api/users/{id}", o.getUserId()).retrieve().bodyToMono(UserDto.class));

        return Mono.zip(order, product, user)   // fan-out, fan-in — all 3 calls run concurrently
            .map(tuple -> new OrderFullView(tuple.getT1(), tuple.getT2(), tuple.getT3()));
    }
}
```

Real-Time Usage

- **Kubernetes-native systems**: often skip Eureka entirely and use K8s's built-in DNS-based service discovery + `Service` objects — same concept, platform-provided.
- **Aggregator services** are common for dashboard/report endpoints (e.g., "customer 360 view" combining CRM, billing, and support-ticket services into one API call for a support agent's screen).

Architectural Flow Diagram

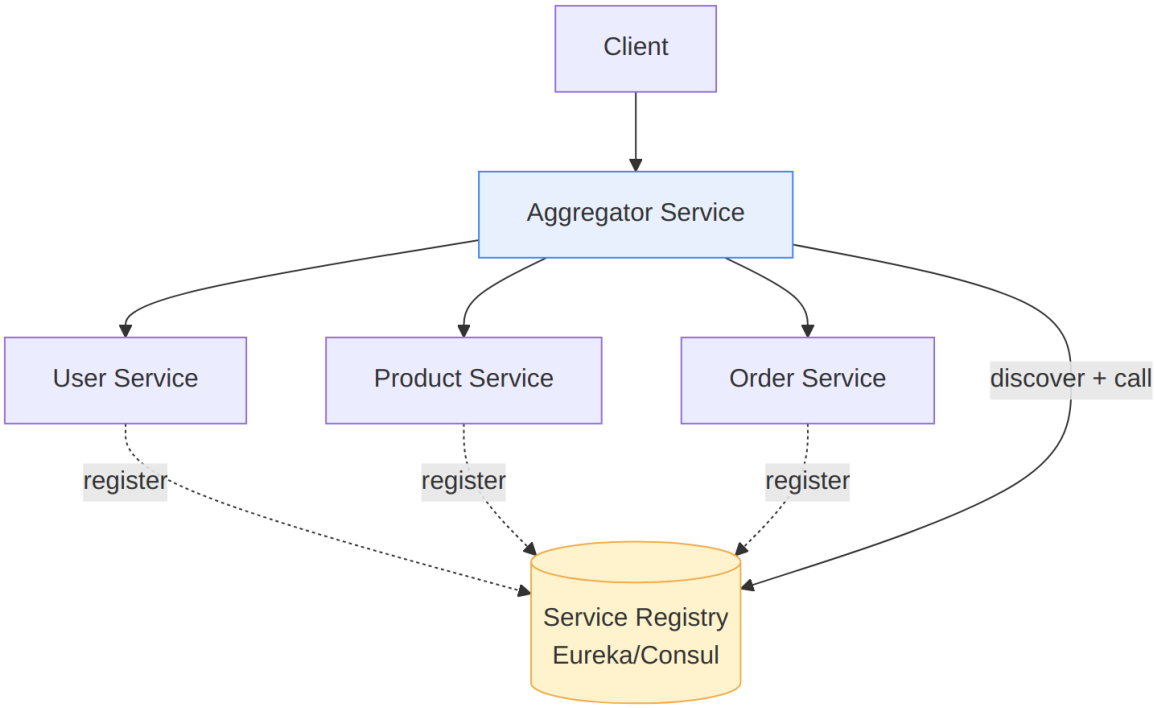


Figure 29: Architectural Flow Diagram

Q6: Microservices Architecture — Message Broker

Definition

A **Message Broker** decouples producers and consumers, providing durability, delivery guarantees, and buffering. Two broad models:

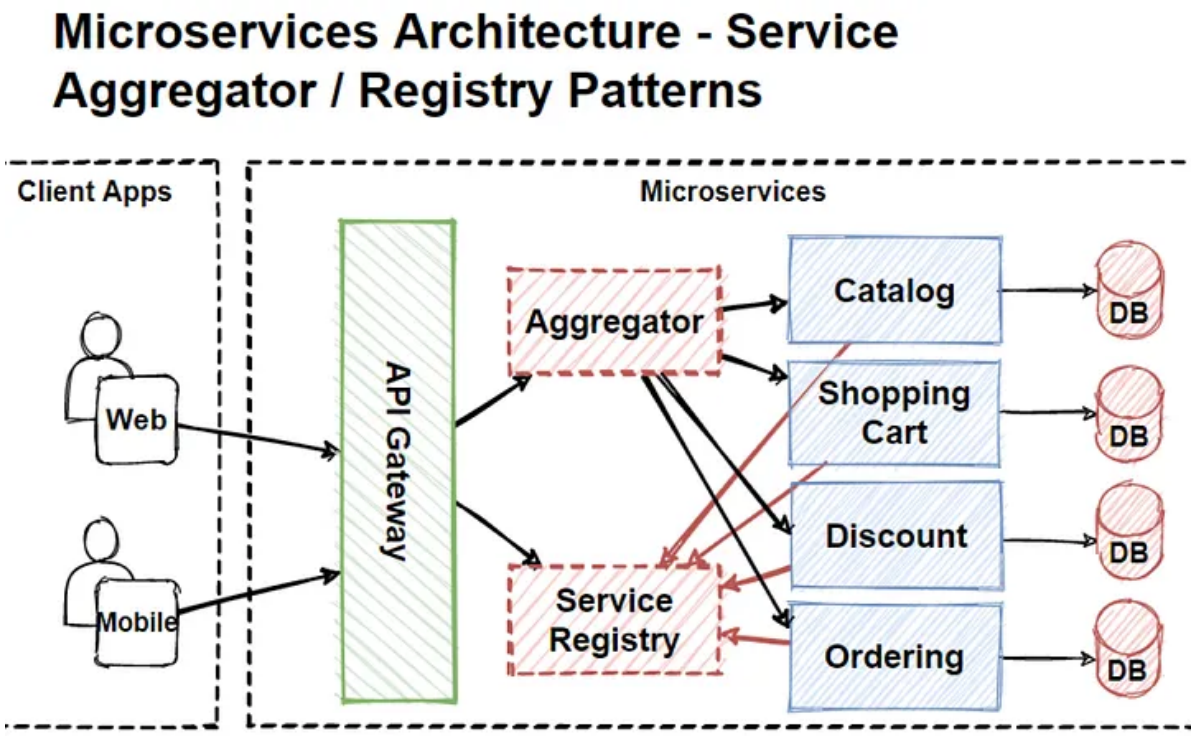


Figure 30: Microservices Architecture with Service Aggregator / Registry Patterns — Web and Mobile clients through an API Gateway to an Aggregator, which discovers and calls Catalog, Shopping Cart, Discount, and Ordering via a Service Registry

- **Point-to-Point (Queue)**: one message consumed by exactly one consumer (work-queue / task distribution) — classic RabbitMQ queue.
- **Publish-Subscribe (Topic)**: one message delivered to *all* subscribed consumer groups — Kafka topics, RabbitMQ fanout/topic exchanges.

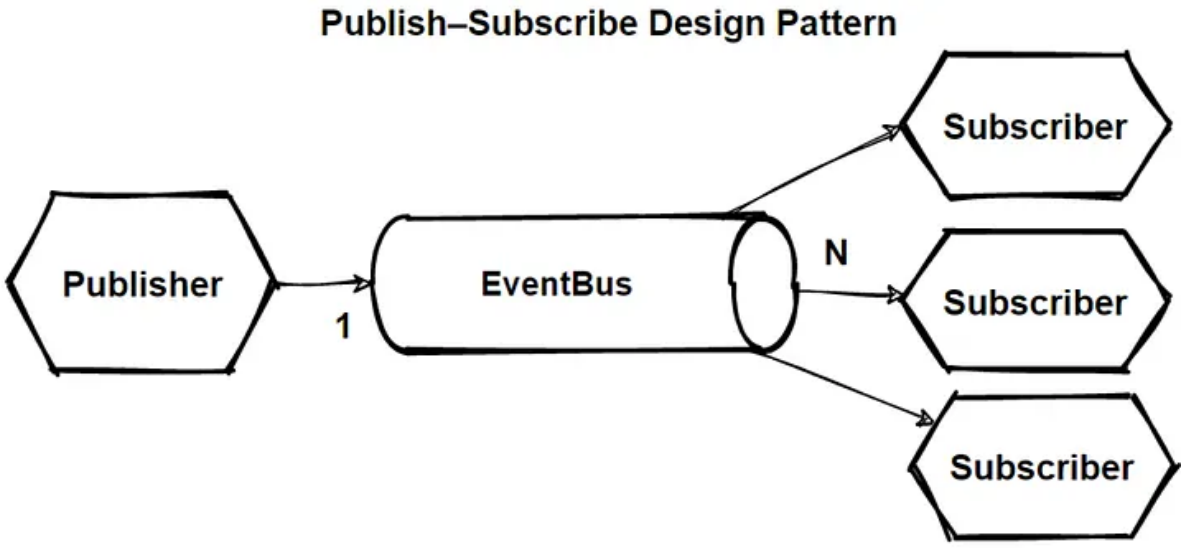


Figure 31: Publish-Subscribe Design Pattern — a Publisher sends one message to an EventBus, which fans it out to N Subscribers

	Kafka	RabbitMQ
Model	Log-based, pub-sub, replayable	Traditional queue broker, point-to-point + pub-sub
Throughput	Very high, built for streaming	High, better for complex routing
Message retention	Configurable (hours→forever), consumers control offset	Deleted once acked (unless using streams)
Best for	Event sourcing, high-volume event streams	Task queues, RPC-style patterns, complex routing rules

Hands-On: RabbitMQ with Spring AMQP (complementing the Kafka setup from Part II)

```
@Configuration
public class RabbitConfig {
    @Bean public Queue emailQueue() { return new Queue("email.queue", true); }
    @Bean public TopicExchange notificationExchange() { return new TopicExchange("notification.exchange"); }

    @Bean
    public Binding binding(Queue emailQueue, TopicExchange notificationExchange) {
        return BindingBuilder.bind(emailQueue).to(notificationExchange).with("order.confirmed.*");
    }
}
```

```java
@Service
public class NotificationProducer {
    private final RabbitTemplate rabbitTemplate;
    public void notifyOrderConfirmed(String orderId, String email) {
        rabbitTemplate.convertAndSend("notification.exchange", "order.confirmed.email",
                new EmailNotification(orderId, email));
    }
}

@Component
public class EmailConsumer {
    @RabbitListener(queues = "email.queue")
    public void onMessage(EmailNotification notification) {
        emailService.send(notification.email(), "Your order " + notification.orderId() + " is confirmed!");
        // throwing here -> message requeued or routed to a Dead Letter Exchange per config
    }
}
```

```yaml
spring:
  rabbitmq:
    listener:
      simple:
        retry:
          enabled: true
          max-attempts: 3
        default-requeue-rejected: false   # send to DLX after max-attempts instead of infinite requeue
```

Real-Time Usage

- **Kafka**: order events, clickstream/analytics pipelines, log aggregation — anywhere you need replay and very high throughput (recap: Part II Q2).
- **RabbitMQ**: task queues like "send email," "generate PDF invoice," "resize image" — classic worker-queue patterns with complex routing (topic exchanges by region/priority).

Architectural Flow Diagram

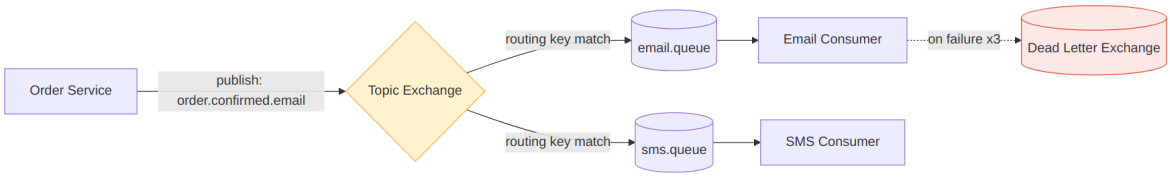


Figure 32: Architectural Flow Diagram

Q7: Microservices Architecture — Database Sharding (Cassandra)

Definition

Sharding (partitioning) splits a large dataset across multiple nodes so no single node holds all the data. Cassandra shards **automatically** using **consistent hashing** on the **partition key**: each row's partition key is hashed to a token, and the token determines which node(s) in the ring own that data (replicated to `N` nodes per the **replication factor**).

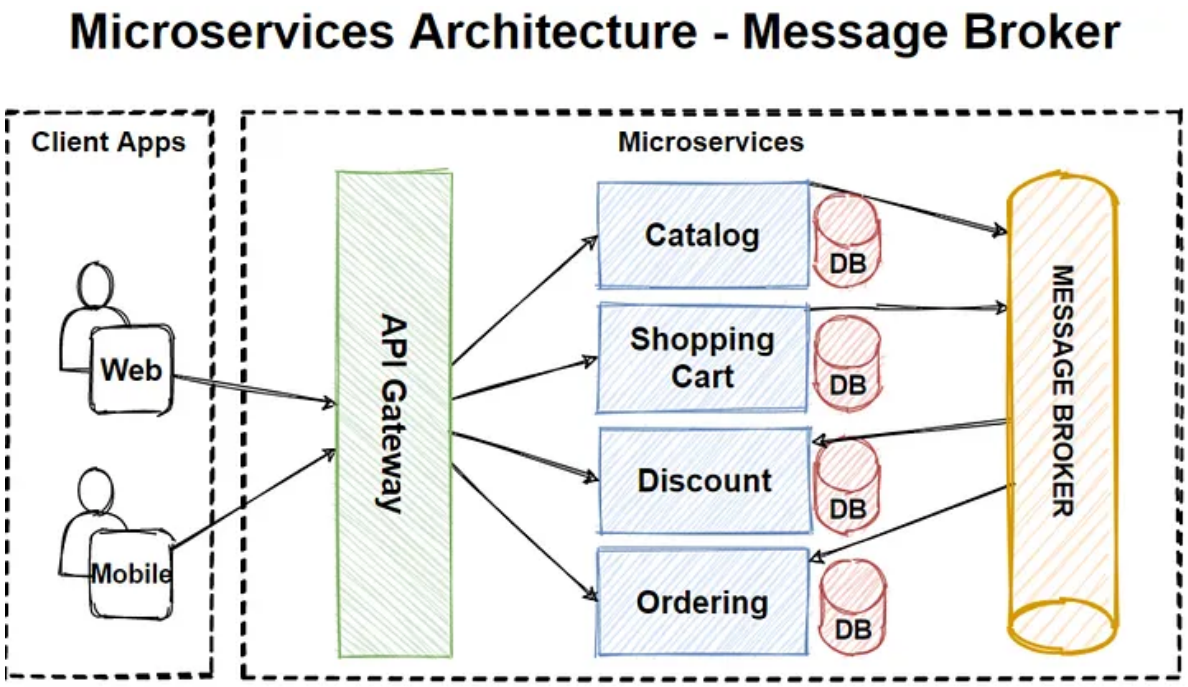


Figure 33: Microservices Architecture with a Message Broker — Web and Mobile clients through an API Gateway to Catalog, Shopping Cart, Discount, and Ordering, each with its own DB, all publishing/consuming through a central Message Broker

Good partition-key design is the single most important Cassandra decision — it determines the “shard” your query hits.

Hands-On: Designing a Sharded Order Table

```
-- BAD: single global partition ("status") -> massive hot partition on one node
CREATE TABLE orders_bad (
    status text,
    order_id uuid,
    PRIMARY KEY (status, order_id)
);

-- GOOD: partition by user_id (naturally distributes across the ring),
-- cluster by order date descending for fast "recent orders" queries
CREATE TABLE orders_by_user (
    user_id     bigint,
    order_date  timestamp,
    order_id    uuid,
    product_id  bigint,
    status      text,
    total       decimal,
    PRIMARY KEY ((user_id), order_date, order_id)
) WITH CLUSTERING ORDER BY (order_date DESC);
```

```
@Query("SELECT * FROM orders_by_user WHERE user_id = ?0 LIMIT 20")
List<OrderEntity> findRecentOrders(Long userId);  // hits exactly ONE partition/shard — fast, scalable
```

```
# Keyspace replication — 3 replicas across a multi-DC ring
CREATE KEYSPACE ecommerce WITH replication = {
  'class': 'NetworkTopologyStrategy',
  'dc1': 3,
  'dc2': 3
};
```

```
// Consistency level tunes the CAP trade-off per query
cqlSession.execute(SimpleStatement.newInstance(
    "SELECT * FROM orders_by_user WHERE user_id = ?", userId)
    .setConsistencyLevel(ConsistencyLevel.LOCAL_QUORUM));  // majority of local-DC replicas must respond
```

Real-Time Usage

- **Order history / activity feeds at scale** (billions of rows): Cassandra's partition-per-user model keeps writes and reads O(1) regardless of total dataset size, unlike a single sharded-by-hash-only relational table.
- **IoT time-series data**: partition by `device_id`, cluster by `timestamp` — the same pattern as orders, applied to sensor readings.
- **Netflix's viewing-history and Instagram's inbox systems** use this exact "partition by owner entity, cluster by time" Cassandra pattern for near-linear horizontal scalability.

Architectural Flow Diagram

Q8: Event-Driven Microservices Architecture (Recap + Transactional Outbox)

Definition — Recap

Services communicate primarily via **events** through a broker rather than direct calls (full definition and Kafka saga hands-on: **Part II Q2**). The one gap not yet solved: the **dual-write problem** — if a service writes to its DB *and* publishes a Kafka event as two separate steps, a crash between them causes inconsistency (DB updated, event lost — or vice versa).

Transactional Outbox Pattern: write the event to an `outbox` table in the **same local DB transaction** as the business data. A separate process (or **Debezium** via Change Data Capture) tails the outbox table and reliably publishes to Kafka — guaranteeing at-least-once delivery consistent with the DB write.

Put simply: when a service's API needs to publish an event, it **doesn't send it directly**. Instead, the event message is **persisted in a database table** as part of the very same transaction that changes the business data. A separate job/process then publishes those persisted events to the message broker at predefined intervals (or reactively, via CDC).

Domain Events are therefore never written straight to an event bus. Instead, they're written to a table playing the "outbox" role inside the service's *own* database. The critical point: **the business transaction and the write to the outbox table are part of the same atomic transaction** — so either both happen, or neither does. For example, when a new product is added, writing the `Product` row and writing the `ProductCreated` event to the outbox happen together, guaranteeing the event is never lost even if the service crashes immediately after committing.

The second step is a separate, independent process that reads the events written to the outbox table and publishes them onto the event bus — decoupling "recording that something

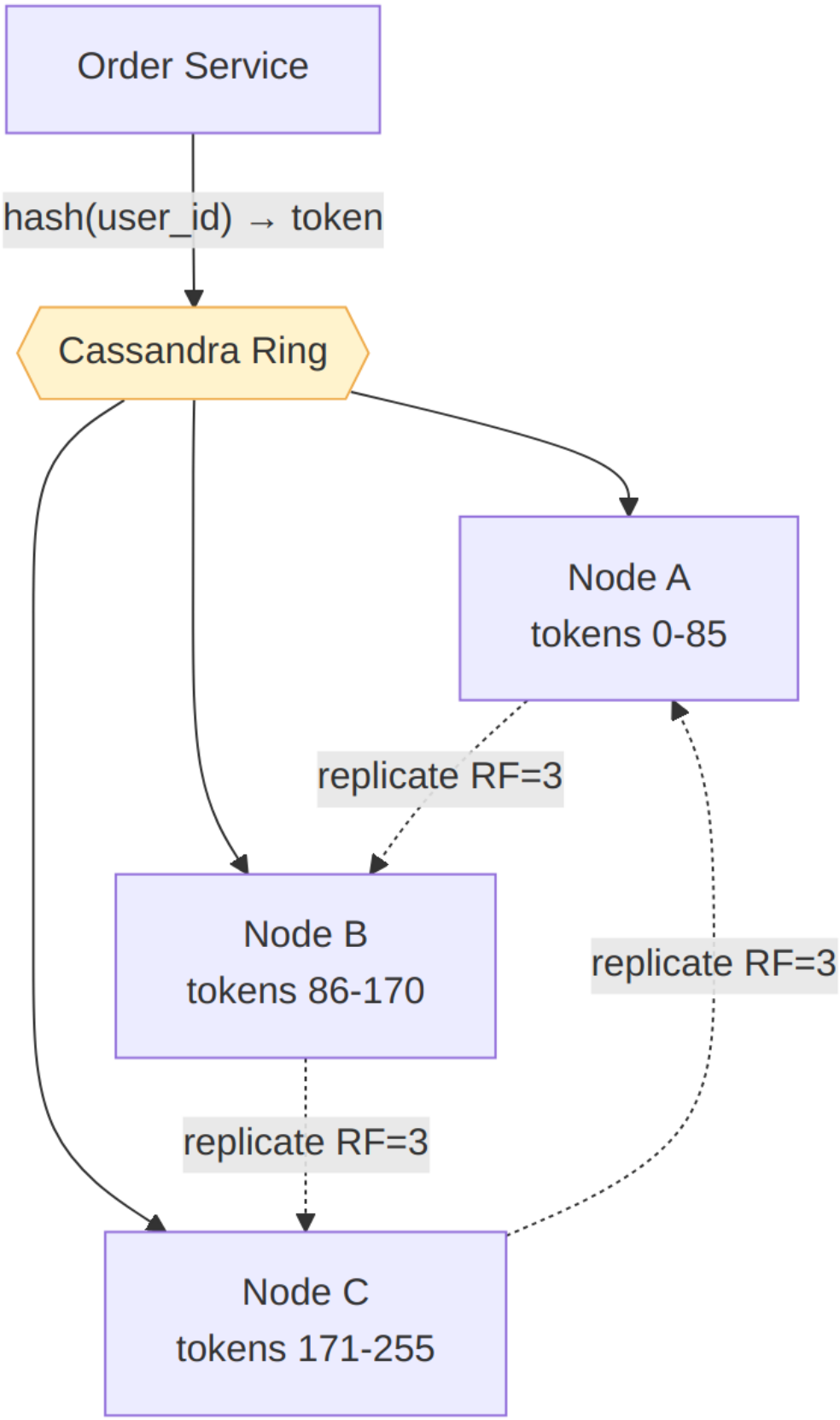


Figure 34: Architectural Flow Diagram

happened” from “reliably telling everyone else about it.”

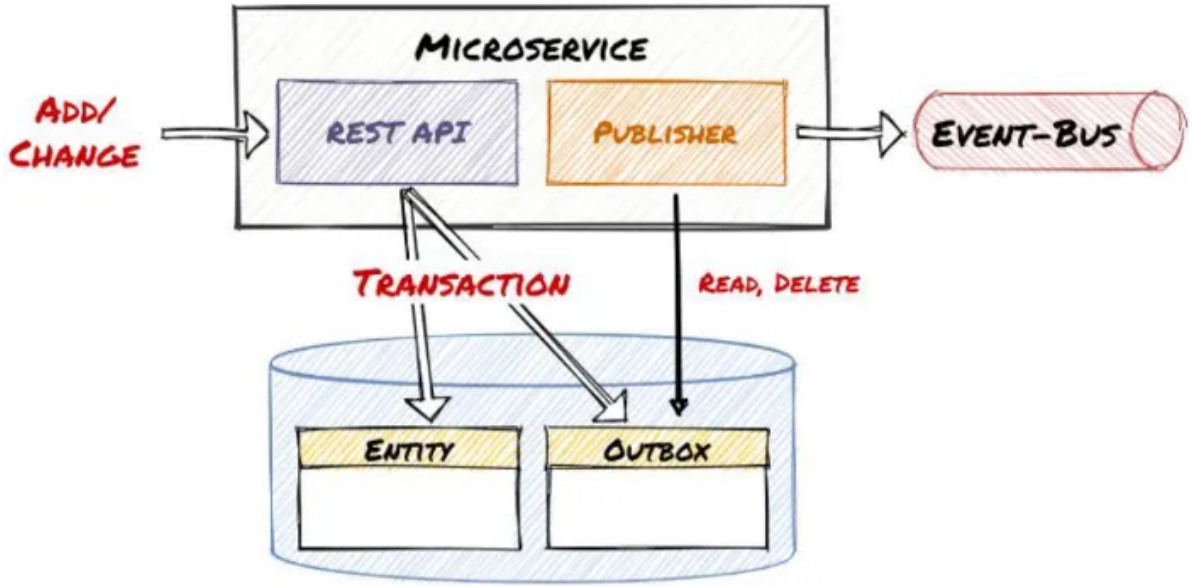


Figure 35: The Outbox Pattern — an Add/Change request hits the microservice’s REST API, which writes to the Entity table and the Outbox table in one Transaction; a separate Publisher reads/deletes from the Outbox and forwards events to the Event-Bus

Why use the Outbox Pattern? Whenever you’re working with critical data that must stay consistent and must capture every request accurately, the outbox pattern is the right tool — anywhere the **database update and the message send must be atomic** to guarantee data consistency. Order/sale transactions are the textbook case: since they’re financial, the numbers must be correct 100% of the time, which means the system must never silently lose an event message. This is exactly the accuracy guarantee the Outbox Pattern provides.

Hands-On

```
CREATE TABLE outbox (
    id UUID PRIMARY KEY,
    aggregate_type VARCHAR(50),
    aggregate_id VARCHAR(50),
    event_type VARCHAR(50),
    payload JSONB,
    created_at TIMESTAMP DEFAULT now()
);
```

```
@Service
public class OrderService {

    @Transactional   // business write + outbox write in ONE atomic DB transaction
    public Order placeOrder(CreateOrderCommand cmd) {
        Order order = orderRepository.save(new Order(cmd));

        outboxRepository.save(new OutboxEvent(
            UUID.randomUUID(), "Order", order.getId(), "OrderPlaced",
            toJson(new OrderPlacedEvent(order))));
```

```
        return order;   // no direct Kafka call here — Debezium/relay handles delivery
    }
}
```

```
// Debezium connector config — tails the outbox table's WAL/binlog, publishes to Kafka automatically
{
  "name": "order-outbox-connector",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "table.include.list": "public.outbox",
    "transforms": "outbox",
    "transforms.outbox.type": "io.debezium.transforms.outbox.EventRouter",
    "topic.creation.default.topic": "order.events"
  }
}
```

Real-Time Usage

- **Any financial or order system** where losing an event (e.g., "PaymentCompleted") silently would cause data loss — outbox + Debezium is the industry-standard fix, used at Shopify, Uber, and most fintechs running Kafka.

Architectural Flow Diagram

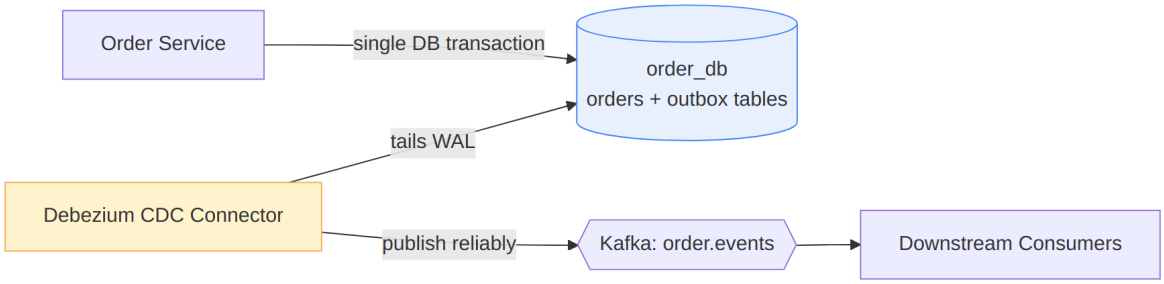


Figure 36: Architectural Flow Diagram

Q9: Microservices Distributed Caching (Recap + Redis Cluster / Distributed Lock)

Definition — Recap

Two-level Caffeine (L1) + Redis (L2) caching with `@Cacheable`/`@CachePut`/`@CacheEvict` and LRU/LFU/TTL eviction was fully covered in **Part III Q4**. New addition here: making Redis itself **highly available** and using it for **distributed locking** (a common companion need in caching-heavy microservices — e.g., preventing a cache stampede).

Hands-On: Redis Cluster HA + Distributed Lock (Redisson) to prevent cache stampede

```
spring:
  redis:
```

```
    cluster:
      nodes: redis-node1:6379,redis-node2:6379,redis-node3:6379
      max-redirects: 3
```

```
@Service
public class ProductService {

    private final RedissonClient redisson;
    private final ProductRepository repository;

    // Prevents "thundering herd": only ONE instance recomputes on cache-miss, others wait then read cache
    public ProductDto getProduct(Long id) {
        String cacheKey = "product:" + id;
        ProductDto cached = redisCache.get(cacheKey);
        if (cached != null) return cached;

        RLock lock = redisson.getLock("lock:" + cacheKey);
        lock.lock(5, TimeUnit.SECONDS);
        try {
            cached = redisCache.get(cacheKey);          // double-check after acquiring lock
            if (cached != null) return cached;
            ProductDto fresh = ProductMapper.toDto(repository.findById(id).orElseThrow());
            redisCache.put(cacheKey, fresh, Duration.ofMinutes(30));
            return fresh;
        } finally {
            lock.unlock();
        }
    }
}
```

Real-Time Usage

- **Flash-sale product pages**: without a stampede lock, a popular item's cache expiring under massive concurrent load can send thousands of simultaneous queries to the DB at once — the distributed lock ensures only one recomputes it.

Architectural Flow Diagram

Q10: Microservices Containers and Orchestrators with CI/CD (Recap + Autoscaling & Helm)

Definition — Recap

Docker packaging, Kubernetes Deployments, and a full GitHub Actions CI/CD pipeline with Canary/Blue-Green were fully covered in **Part III Q5**. New addition: **Horizontal Pod Autoscaling (HPA)** and packaging deployments as **Helm charts** for repeatable, parameterized releases.

Hands-On

```
# Horizontal Pod Autoscaler — scale order-service based on CPU AND custom Kafka-lag metric
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
```

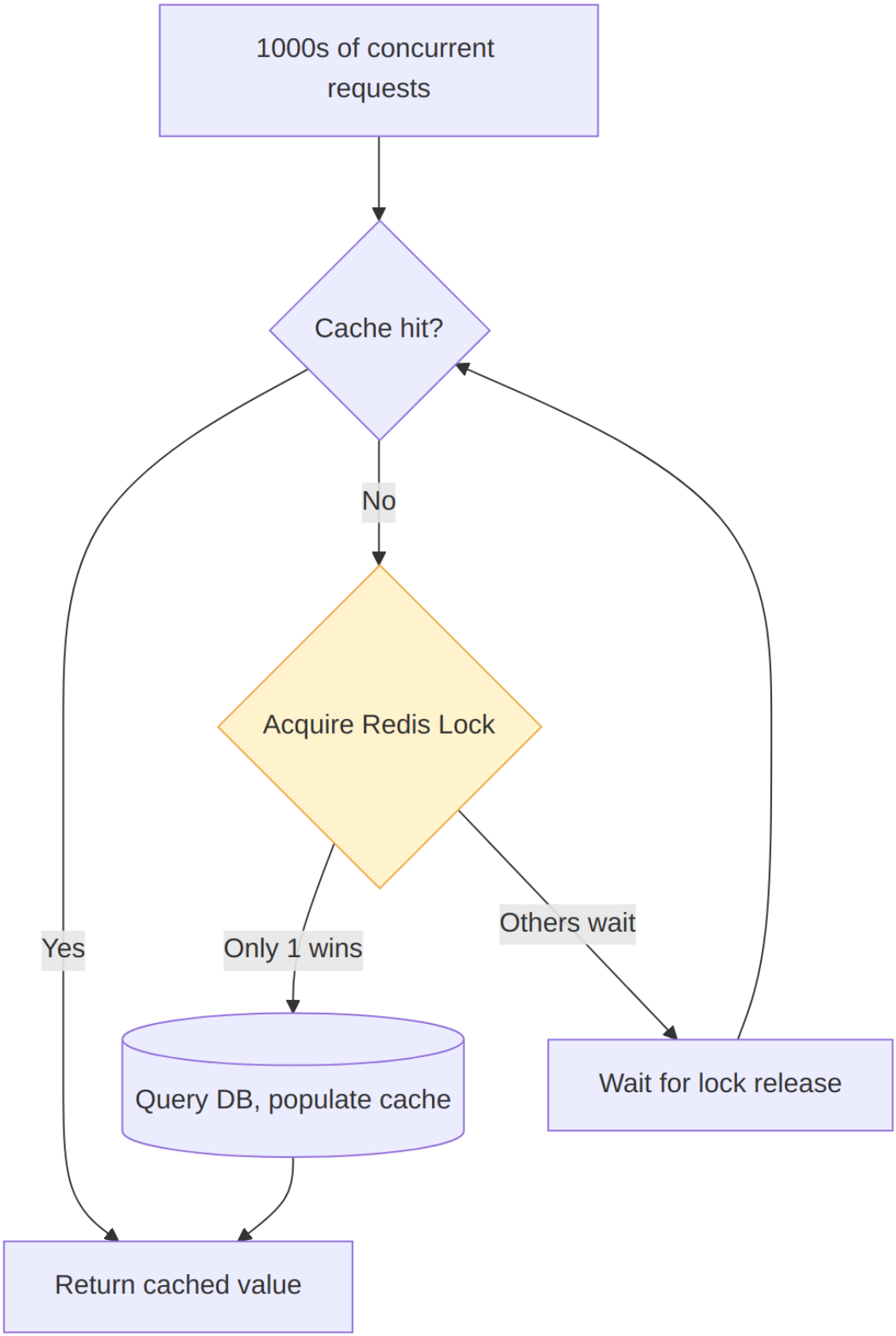


Figure 37: Architectural Flow Diagram

```yaml
metadata: { name: order-service-hpa }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: order-service-v1 }
  minReplicas: 3
  maxReplicas: 30
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 65 } }
    - type: Pods
      pods:
        metric: { name: kafka_consumergroup_lag }
        target: { type: AverageValue, averageValue: "1000" }
```

```
# Helm chart structure — parameterize the same manifests across environments
order-service-chart/
 ├── Chart.yaml
 ├── values-dev.yaml       # replicas: 1, resources: small
 ├── values-staging.yaml   # replicas: 3
 ├── values-prod.yaml      # replicas: 10, HPA enabled
 └── templates/
      ├── deployment.yaml   # {{ .Values.replicas }}, {{ .Values.image.tag }}
      ├── service.yaml
      └── hpa.yaml
```

```
helm upgrade --install order-service ./order-service-chart -f values-prod.yaml --set image.tag=$GIT_SHA
```

Real-Time Usage

- **Autoscaling on Kafka consumer lag** (not just CPU) is standard for event-driven services — CPU can look idle while a backlog builds up in the queue; KEDA/custom-metrics HPA scales on the metric that actually matters.
- **Helm** is the de-facto packaging standard for multi-environment Kubernetes deployments across virtually all cloud-native enterprises.

Architectural Flow Diagram

Q11: Microservices Resilience4j — Circuit Breaker, Retry, Rate Limiter, Bulkhead

Definition

Resilience4j is a lightweight fault-tolerance library providing four complementary patterns, typically stacked together:

Pattern	Purpose	Key mechanism
Circuit Breaker	Stop calling a failing downstream service to let it recover	State machine: `CLOSED` → `OPEN` → `HALF_OPEN` based on failure rate
Retry	Recover from transient/blip failures	Configurable attempts + backoff (fixed, exponential, jittered)

Pattern	Purpose	Key mechanism
Rate Limiter	Cap outgoing/incoming call rate	Algorithms: fixed-window counter (Resilience4j's core impl), can also front with token-bucket (gateway)
Bulkhead	Isolate failures so one slow dependency can't exhaust all threads	Semaphore-based (limit concurrent calls) or ThreadPool-based (dedicated thread pool per dependency)

Hands-On: Full Stack — Circuit Breaker + Retry + Rate Limiter + Bulkhead on the Feign call to `product-service`

```
<dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-spring-boot3</artifactId>
</dependency>
```

```
resilience4j:
  circuitbreaker:
    instances:
      productService:
        slidingWindowType: COUNT_BASED
        slidingWindowSize: 20
        failureRateThreshold: 50          # open circuit if >=50% of last 20 calls fail
        waitDurationInOpenState: 10s       # stay OPEN for 10s before trying HALF_OPEN
        permittedNumberOfCallsInHalfOpenState: 5

  retry:
    instances:
      productService:
        maxAttempts: 3
        waitDuration: 500ms
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2     # 500ms -> 1s -> 2s
        retryExceptions: [java.io.IOException, org.springframework.web.client.ResourceAccessException]

  ratelimiter:
    instances:
      productService:
        limitForPeriod: 50           # fixed-window algorithm: 50 permits per refresh period
        limitRefreshPeriod: 1s
        timeoutDuration: 100ms       # how long a call waits for a permit before failing fast

  bulkhead:                          # semaphore-based: caps CONCURRENT calls
    instances:
      productService:
        maxConcurrentCalls: 20
        maxWaitDuration: 50ms

  thread-pool-bulkhead:              # thread-pool-based: full isolation, own executor
    instances:
      productServiceHeavy:
        maxThreadPoolSize: 10
        coreThreadPoolSize: 5
        queueCapacity: 20
```

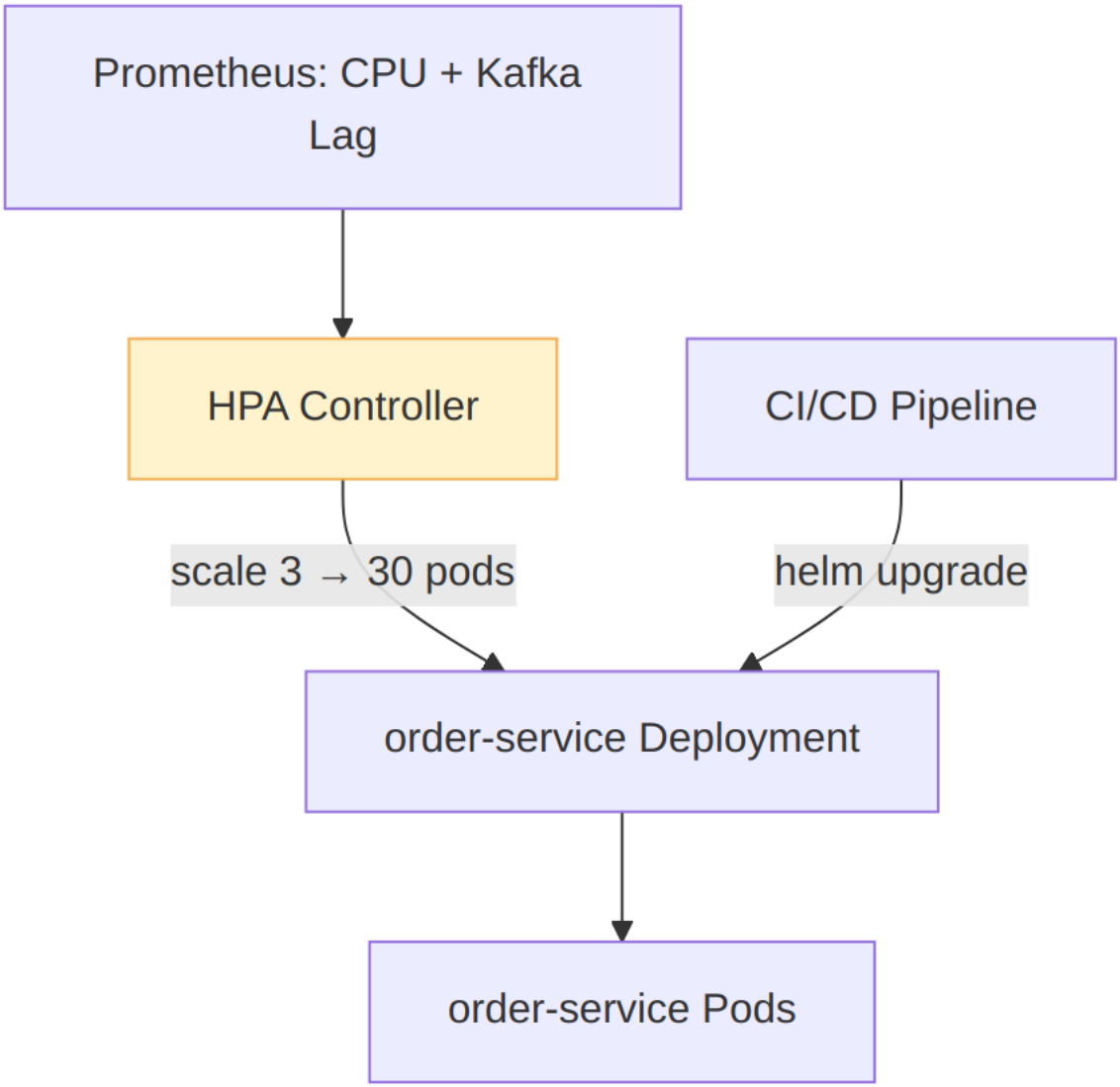


Figure 38: Architectural Flow Diagram

```java
@Service
public class ProductLookupService {

    private final ProductClient productClient;  // Feign client from Part III Q2

    @CircuitBreaker(name = "productService", fallbackMethod = "fallbackProduct")
    @Retry(name = "productService")
    @RateLimiter(name = "productService")
    @Bulkhead(name = "productService")
    public ProductDto getProduct(Long id) {
        return productClient.getProduct(id);   // real call — protected by all 4 patterns, in order
    }

    // Fallback triggers on: circuit OPEN, retries exhausted, rate-limit exceeded, or bulkhead full
    private ProductDto fallbackProduct(Long id, Throwable t) {
        log.warn("Falling back for product {} due to: {}", id, t.toString());
        return ProductDto.unavailable(id);   // degrade gracefully instead of propagating failure
    }
}
```

```java
// ThreadPool bulkhead variant — for a genuinely slow/heavy dependency needing full thread isolation
@Bulkhead(name = "productServiceHeavy", type = Bulkhead.Type.THREADPOOL, fallbackMethod = "fallbackAsync")
public CompletableFuture<ProductDto> getProductHeavy(Long id) {
    return CompletableFuture.supplyAsync(() -> productClient.getFullDetails(id));
}
```

```java
// Observing circuit breaker state transitions (feeds into Metrics/Grafana in Part V)
circuitBreakerRegistry.circuitBreaker("productService")
    .getEventPublisher()
    .onStateTransition(event -> log.info("CB state: {} -> {}",
        event.getStateTransition().getFromState(), event.getStateTransition().getToState()));
```

Hands-On: Applying Resilience4j at the API Gateway Level (`apigateway.yaml`)

The example above protects a single service-to-service call. Just as important is applying these same four patterns **at the gateway itself**, using Spring Cloud Gateway's `resilience4j-reactor-filter` — so every client request into `order-service` is protected centrally, without every downstream client needing its own copy of this config.

```xml
<dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-spring-cloud2</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
```

a) Circuit Breaker — All States + `apigateway.yaml` Config Resilience4j's Circuit Breaker is a **state machine** with three main operating states (plus two special override states):

State	Behavior
CLOSED	Normal operation. Every call passes through to `order-service`. Successes/failures are recorded in a sliding window.
OPEN	Tripped. The failure/slow-call rate exceeded the threshold. **All calls fail fast** (no network call made) for `waitDurationInOpenState`, giving the downstream service room to recover.
HALF_OPEN	After the wait duration elapses, a limited number of **trial calls** are permitted through. Enough successes → back to `CLOSED`. Still failing → back to `OPEN`.
DISABLED *(override)*	Circuit breaker logic is switched off entirely — every call passes through, no state tracking. Useful for local dev/testing.
FORCED_OPEN *(override)*	Manually forced to always fail fast, ignoring metrics — useful for planned maintenance windows on a downstream dependency.

State transition rules, precisely: - **CLOSED → OPEN**: once at least `minimumNumberOfCalls` have been recorded in the sliding window, **and** either the failure rate or the slow-call rate exceeds its threshold. - **OPEN → HALF_OPEN**: automatically after `waitDurationInOpenState` (if `automaticTransitionFromOpenToHalfOpenEnabled: true`), otherwise on the next incoming call after the wait elapses. - **HALF_OPEN → CLOSED**: if the trial calls permitted in `permittedNumberOfCallsInHalfOpenState` succeed at or above the configured threshold. - **HALF_OPEN → OPEN**: if those trial calls still fail at or above the threshold — the circuit re-trips without ever fully closing.

```yaml
# apigateway.yaml — Circuit Breaker fully configured at the gateway level, all states covered
spring:
  cloud:
    gateway:
      routes:
        - id: order-service-route
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - name: CircuitBreaker
              args:
                name: orderServiceCB
                fallbackUri: forward:/fallback/orders

resilience4j:
  circuitbreaker:
    configs:
      default:
        registerHealthIndicator: true
        slidingWindowType: COUNT_BASED                  # or TIME_BASED (window measured in seconds)
        slidingWindowSize: 20                            # evaluate the last 20 calls
        minimumNumberOfCalls: 10                         # don't judge failure rate until 10 calls seen
        failureRateThreshold: 50                         # CLOSED -> OPEN if >=50% of calls fail
       slowCallRateThreshold: 80                        # calls slower than the line below also count as "failed"
        slowCallDurationThreshold: 2s
        waitDurationInOpenState: 10s                     # OPEN -> HALF_OPEN after this wait
        permittedNumberOfCallsInHalfOpenState: 5         # trial calls allowed while HALF_OPEN
```

```
        automaticTransitionFromOpenToHalfOpenEnabled: true
        recordExceptions:
          - java.io.IOException
          - org.springframework.web.client.ResourceAccessException
    instances:
      orderServiceCB:
        baseConfig: default
```

```
// Fallback endpoint the gateway forwards to whenever the circuit is OPEN or HALF_OPEN trial calls fail
@RestController
public class FallbackController {
    @GetMapping("/fallback/orders")
    public ResponseEntity<Map<String, Object>> orderFallback() {
        return ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE)
            .body(Map.of("message", "Order service is temporarily unavailable — please retry shortly."));
    }
}
```

b) Retry — Exponential Backoff at the Gateway The exponential backoff formula Resilience4j uses: `wait_n = min(initialWait × multiplier^(n-1), maxWaitDuration)`, optionally with random jitter added to avoid many clients retrying in lockstep ("thundering herd").

```
# apigateway.yaml — Retry with exponential backoff, wired to the same route
spring:
  cloud:
    gateway:
      routes:
        - id: order-service-route
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - name: Retry
              args:
                retries: 4
                statuses: BAD_GATEWAY, SERVICE_UNAVAILABLE, GATEWAY_TIMEOUT
                methods: GET, POST
                backoff:
                  firstBackoff: 500ms
                  maxBackoff: 5s
                  factor: 2                # 500ms -> 1s -> 2s -> 4s (capped at maxBackoff)
                  basedOnPreviousValue: false

resilience4j:
  retry:
    configs:
      default:
        maxAttempts: 4
        waitDuration: 500ms
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2       # each retry waits 2x longer than the previous one
        exponentialMaxWaitDuration: 5s        # cap on how long a single backoff can grow to
        enableRandomizedWait: true            # add jitter so many clients don't retry in the same instant
        randomizedWaitFactor: 0.5
        retryExceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
```

```
    instances:
      orderServiceRetry:
        baseConfig: default
```

```
// Service-level equivalent, for comparison — same exponential backoff config applied via annotation
@Retry(name = "orderServiceRetry", fallbackMethod = "retryFallback")
public OrderDto getOrder(String orderId) {
    return orderClient.getOrder(orderId);
}

private OrderDto retryFallback(String orderId, Throwable t) {
    log.warn("All retry attempts exhausted for order {}: {}", orderId, t.toString());
    throw new OrderServiceUnavailableException(orderId);
}
```

c) Rate Limiter — Token Bucket Algorithm at the Gateway Spring Cloud Gateway's built-in `RequestRateLimiter` filter implements a **Token Bucket** algorithm (backed by Redis, shared cluster-wide across every gateway instance): a bucket holds up to `burstCapacity` tokens, refills at `replenishRate` tokens/sec, and each request consumes `requestedTokens`. A request is allowed only if enough tokens are currently available in the bucket — this smooths bursts while still enforcing a steady average rate.

```
# apigateway.yaml — Token Bucket rate limiting (Redis-backed, cluster-wide)
spring:
  cloud:
    gateway:
      routes:
        - id: order-service-route
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 50    # steady-state: 50 tokens added per second
                redis-rate-limiter.burstCapacity: 100     # bucket size: allows short bursts up to 100
                redis-rate-limiter.requestedTokens: 1     # each request costs 1 token
                key-resolver: "#{@userKeyResolver}"       # rate-limit per user, not globally

# Alternative: Resilience4j's own rate limiter (fixed-window-counter algorithm) for non-gateway call sites
resilience4j:
  ratelimiter:
    configs:
      default:
        limitForPeriod: 50            # 50 permits...
        limitRefreshPeriod: 1s         # ...replenished every 1 second (fixed window, not a rolling bucket)
        timeoutDuration: 200ms         # how long a call waits for a permit before failing fast
    instances:
      orderServiceRL:
        baseConfig: default
```

```
@Bean
public KeyResolver userKeyResolver() {
    // Buckets are per-user (X-User-Id header) rather than one shared bucket for all clients
    return exchange -> Mono.just(
        Optional.ofNullable(exchange.getRequest().getHeaders().getFirst("X-User-Id"))
            .orElse("anonymous"));
```

```
}
```

```yaml
# apigateway.yaml — Bulkhead, both semaphore (limit concurrency) and thread-pool (full isolation) variants
resilience4j:
  bulkhead:                                  # semaphore-based: caps concurrent in-flight calls
    instances:
      orderServiceBulkhead:
        maxConcurrentCalls: 25
        maxWaitDuration: 50ms

  thread-pool-bulkhead:                      # thread-pool-based: dedicated executor, full isolation
    instances:
      orderServiceHeavyBulkhead:
        maxThreadPoolSize: 10
        coreThreadPoolSize: 5
        queueCapacity: 20
        keepAliveDuration: 20ms

spring:
  cloud:
    gateway:
      routes:
        - id: order-service-route
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - name: Bulkhead
              args:
                name: orderServiceBulkhead
```

```java
// Custom GatewayFilterFactory — Spring Cloud Gateway has no built-in "Bulkhead" filter,
// so it's wired via Resilience4j's BulkheadRegistry directly at the gateway layer
@Component
public class BulkheadGatewayFilterFactory extends
↪  AbstractGatewayFilterFactory<BulkheadGatewayFilterFactory.Config> {

    private final BulkheadRegistry bulkheadRegistry;

    public BulkheadGatewayFilterFactory(BulkheadRegistry bulkheadRegistry) {
        super(Config.class);
        this.bulkheadRegistry = bulkheadRegistry;
    }

    @Override
    public GatewayFilter apply(Config config) {
        Bulkhead bulkhead = bulkheadRegistry.bulkhead(config.getName());
        return (exchange, chain) -> {
            if (!bulkhead.tryAcquirePermission()) {
                exchange.getResponse().setStatusCode(HttpStatus.TOO_MANY_REQUESTS);
                return exchange.getResponse().setComplete();
            }
            return chain.filter(exchange).doFinally(signal -> bulkhead.onComplete());
        };
    }
```

```
    public static class Config {
        private String name;
        public String getName() { return name; }
        public void setName(String name) { this.name = name; }
    }
}
```

d) Bulkhead — Isolating `order-service` Traffic at the Gateway With all four filters on the same route (`CircuitBreaker`, `Retry`, `RequestRateLimiter`, `Bulkhead`), a single request into `order-service` at the gateway is now protected end-to-end — capped concurrency, rate-limited, retried with backoff on transient errors, and fully isolated from an open circuit — before it ever reaches the service itself.

Real-Time Usage

- **Circuit Breaker**: Netflix Hystrix's spiritual successor — used everywhere a service calls another over the network, to stop hammering an already-struggling dependency (classic example: a payment gateway outage shouldn't cascade into checkout-service threads all hanging).
- **Retry with exponential backoff + jitter**: standard for calling cloud APIs (AWS/GCP SDKs) and any service prone to transient network blips.
- **Rate Limiter**: protecting a third-party API with a strict quota (e.g., an SMS/email provider charging per call, or enforcing a partner's contractual rate limit).
- **Bulkhead**: a slow reporting/analytics dependency gets its **own** thread pool so it can never starve threads needed for the critical checkout path — the classic "ship's bulkhead" analogy: one compartment flooding doesn't sink the whole ship.

Architectural Flow Diagram

Coming Next — Part V

Observability: centralized **Logs** (ELK/Loki), **Metrics** (Prometheus + Grafana), and **Distributed Tracing** (OpenTelemetry + Jaeger) — wiring the circuit-breaker states, cache hit-rates, Kafka lag, and per-service latencies from every part so far into unified dashboards, plus SLO/alerting design. -e

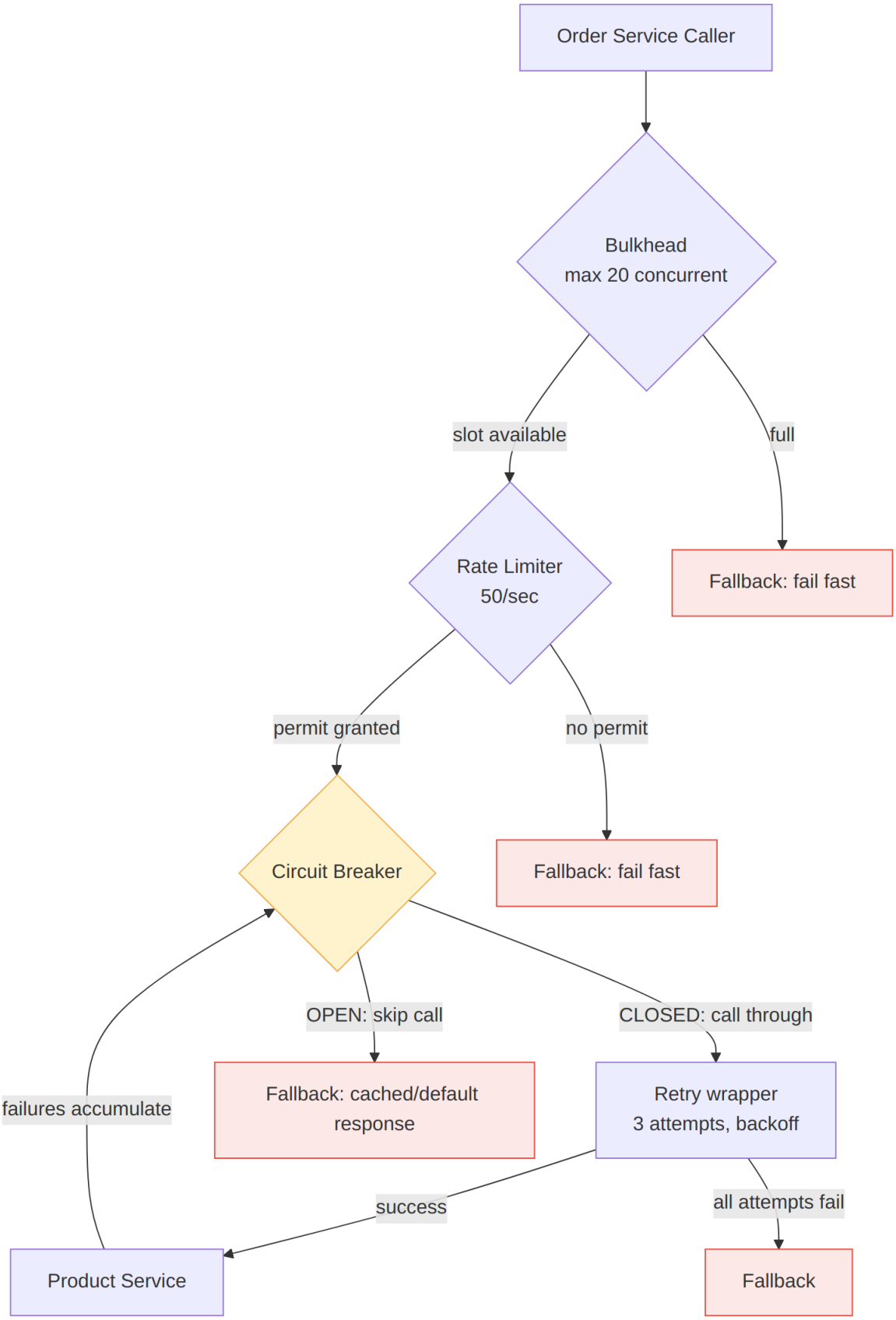


Figure 39: Architectural Flow Diagram